

End-to-End Healthcare Fraud Detection Platform

Full Technical Documentation

Data Warehouse · Machine Learning · ETL .ETL· Cloud · Reporting

Stack: FastAPI · Apache Kafka · Snowflake · dbt · Databricks · Power BI · SSIS · Flask · n8n · Telegram

Team Members:

Amal Abdelmotamed · Shahd Safwat Mohamed · Merna Medhat · Hadeer Badr · Sondos
Abdelmoez

Table of Contents

Table of Contents.....	2
Executive Summary	3
1. Project Overview.....	3
2. Architecture Overview	4
3. Phase 1 — Data Ingestion with FastAPI.....	5
4. Phase 2 — Kafka Streaming	5
5. Phase 3 — Snowflake Cloud Warehouse.....	6
6. Phase 4 — dbt Transformations.....	8
7. Phase 5 — Databricks Medallion Architecture.....	13
8. Phase 6 — Incremental Loading	14
9. Phase 7 — Orchestration	15
10=ssis ,ssas ,dashboard overview	
11. n8n for project Overview	
11.1. Dataset & Features	
11.2. Machine Learning Model	
11.3. API Development	
11.4. Workflow Automation (n8n)	
11.5Telegram Bot.....	
11.6 System Architecture Diagram	
11.7.Results & Discussion.....	
11.8. Future Work	

Executive Summary

This document presents the complete technical documentation for an end-to-end Healthcare Fraud Detection Platform — a multi-phase data engineering project that spans on-premises ingestion, cloud warehousing, dimensional modeling, machine learning, and real-time prediction, all unified around a single business objective: detecting fraudulent healthcare insurance claims within a Medicare-style claims dataset.

The platform was architected in three complementary layers. The first layer is a hybrid data engineering pipeline that ingests raw patient, inpatient, outpatient, and provider fraud data through a FastAPI service, streams it via Apache Kafka to a Snowflake cloud warehouse, and transforms it using dbt into a governed star schema following the Ralph Kimball dimensional modeling methodology — including a central fact table, role-playing date and physician dimensions, junk dimensions for chronic conditions, and bridge tables that correctly resolve the many-to-many relationship between claims and their diagnosis and procedure codes. A parallel Databricks Medallion architecture (Bronze, Silver, Gold) supports heavier data quality validation and advanced analytics workloads alongside the standard BI reporting path. The second layer is a SQL Server-based data warehouse built using SSIS, which applies the same star schema design using on-premises ETL tooling — implementing unpivot transformations for multi-valued diagnosis and procedure columns, lookup transformations for surrogate key resolution, and a role-playing date dimension serving six distinct time-based roles within a single fact table. The third layer is an intelligent real-time fraud screening system: a Gradient Boosting classifier trained on over 650,000 Medicare records (achieving 90.59% test accuracy) is served through a Flask REST API and made accessible to medical staff through a Telegram chatbot automated by an n8n workflow, enabling instant fraud probability scoring for individual claims without manual data entry into a traditional interface.

Taken together, these three layers form a platform designed around three core engineering principles: separation of concerns (each pipeline stage handled by a purpose-built tool), incremental processing (new claims processed continuously without full dataset reprocessing), and auditability (every transformation step from raw ingestion to the final Power BI dashboard is traceable, testable, and version-controlled).

1. Project Overview

The platform ingests four healthcare claims datasets that mirror the structure of a real-world Medicare claims fraud dataset:

- Patient Data — beneficiary demographics, chronic condition indicators, Medicare Part A/B coverage, and annual reimbursement totals
- Inpatient Claims — hospital admission claims including admission and discharge dates, diagnosis codes, procedure codes, and attending/operating/other physicians

- Outpatient Claims — non-admission claims with a similar structure but without admission/discharge dates
- Fraud Provider Data — a binary label per healthcare provider indicating whether that provider has been flagged for potential fraudulent billing activity

The business objective is to combine these four data sources into a single analytical model that allows fraud analysts to identify patterns associated with fraudulent providers, such as unusually high claim volumes, abnormal reimbursement amounts, irregular diagnosis-procedure combinations, or claims concentrated around specific physicians.

2. Architecture Overview

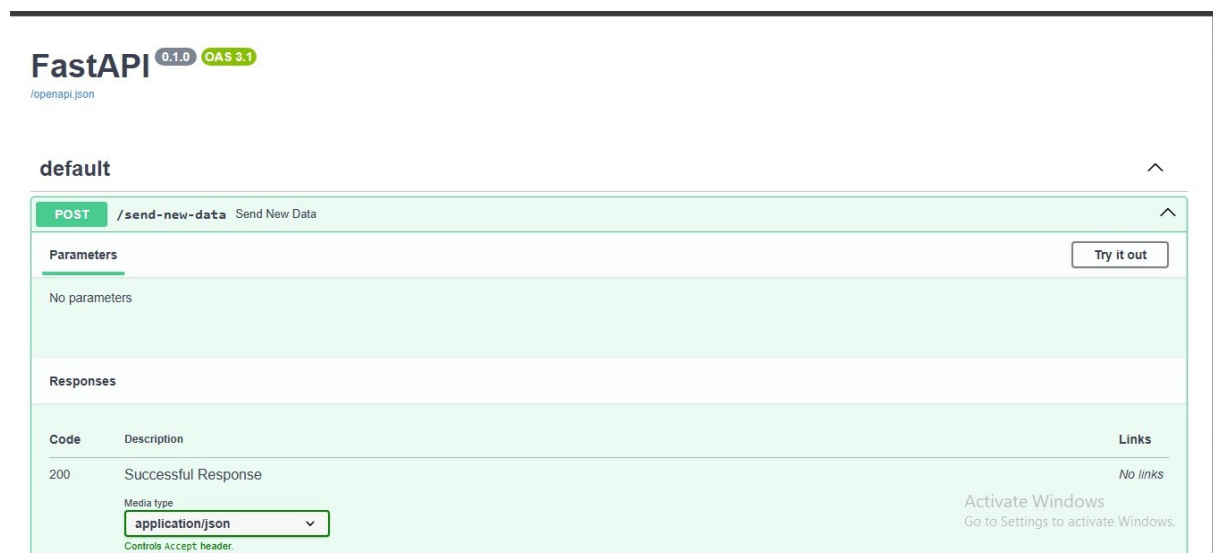
The architecture is intentionally hybrid. Data is ingested and streamed on-premises, then lands in the cloud for storage, transformation, and analytics. The high-level data flow is as follows:

```

FastAPI (On-Prem)
  |
  v
Kafka Producers / Topics (On-Prem)
  |
  v
Kafka Consumers (On-Prem) ---> write_pandas
  |
  v
Snowflake (Cloud) -- Raw / Staging Tables
  |
  v
dbt -- Staging Models -> Dimension Models -> Fact Table -> Bridge Tables
  |
  v
Snowflake -- Star Schema (DWH Layer)
  |
  v
Databricks -- Bronze Layer (raw ingestion)
  |
  v
Databricks -- Silver Layer (cleaned, standardized, validated)
  |
  v
Databricks -- Gold Layer (curated, analytics-ready)
  |
  v
Power BI -- Dashboards and Fraud KPIs
  
```


3. Phase 1 — Data Ingestion with FastAPI

The ingestion layer is built using FastAPI, a Python web framework, exposing HTTP endpoints that read the raw CSV files and publish their contents as JSON messages to Kafka topics.



4. Phase 2 — Kafka Streaming

Kafka was selected as the messaging backbone between ingestion and storage for several engineering reasons:

- Decoupling of producers and consumers: The FastAPI ingestion service does not need to know anything about Snowflake, and the Snowflake-loading consumer does not need to know anything about how the data was generated. They only share a contract: the schema of the JSON messages on each topic.
- Durability and replay: Kafka persists messages on disk for a configurable retention period. If the Snowflake-loading consumer crashes or is taken offline for maintenance, no data is lost — the consumer resumes from its last committed offset when it comes back online.

4.2 Event Streaming and Real-Time Processing Concepts

Event streaming is a data architecture pattern where each unit of data (an event) is published the moment it becomes available, rather than waiting to be processed in scheduled batches. In this platform, each row read from the source CSV is treated as one event — for example, one inpatient claim becoming available is one event published to `inpatient_topic`.

4.3 Topic Design

Four Kafka topics were created, one per data source:

Topic Name	Purpose
patient_topic	Beneficiary demographic and chronic condition records
inpatient_topic	Inpatient claim records
outpatient_topic	Outpatient claim records
provider_topic	Provider-level fraud labels

This one-topic-per-source design was chosen over a single shared topic with a type discriminator field for two reasons. First, each source has a different schema, and Kafka topics are most maintainable when every message on a topic shares a consistent shape. Second, separate topics allow independent scaling and independent consumer logic — the outpatient topic (the largest data source) can be partitioned and consumed differently from the much smaller fraud provider topic without affecting the others.

4.4 Consumer Groups

The Snowflake-loading consumer is registered under a named consumer group (for example, `snowflake_consumer_v1`). Consumer groups in Kafka serve two purposes in this architecture:

- **Offset tracking:** Kafka tracks, per consumer group, the last message offset that has been successfully processed on each topic and partition. This is what allows the consumer to resume exactly where it left off after a restart, rather than reprocessing the entire topic or skipping unprocessed messages.
- **Horizontal scalability:** If claim volume grows, additional consumer instances can be added to the same consumer group. Kafka automatically rebalances topic partitions across the consumers in the group, allowing the system to scale its Snowflake-loading throughput without any code changes.

5. Phase 3 — Snowflake Cloud Warehouse

5.1 Why Snowflake Was Chosen

- **Separation of storage and compute:** Snowflake allows the storage layer (the actual claims data) to scale independently from compute (the warehouses that query it). This means the ingestion warehouse used by the Kafka consumer can be sized and suspended independently from the warehouse used by Power BI or dbt.

- Native semi-structured data support: Snowflake's VARIANT type and native JSON handling made it straightforward to land the JSON payloads coming from Kafka without requiring a rigid pre-defined schema at the raw layer.
- Auto-suspend and auto-resume: Because this is a project-scale workload (the largest table is roughly half a million rows), an XSMALL warehouse configured with AUTO_SUSPEND and AUTO_RESUME keeps compute cost close to zero during idle periods while still being instantly available when a query or load arrives.
- First-class dbt and Python connector support: Snowflake's official Python connector (including the write_pandas utility) and its mature dbt adapter made the Kafka-to-Snowflake and Snowflake-to-dbt boundaries simple to implement reliably.

5.2 Snowflake Architecture in This Project

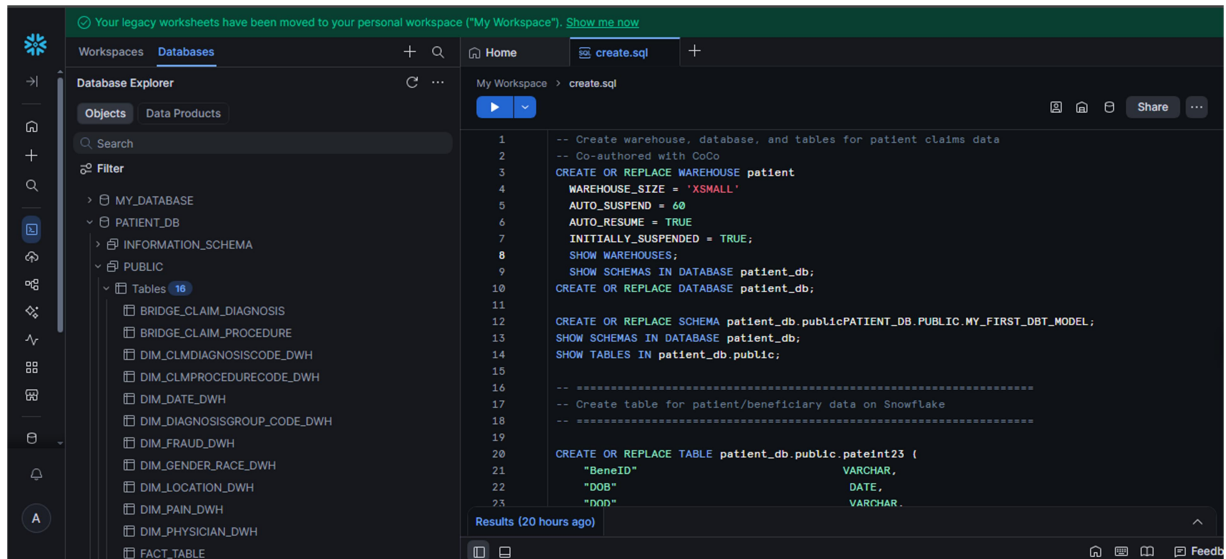
Object	Configuration / Purpose
Warehouse	A single XSMALL warehouse, AUTO_SUSPEND = 60 seconds, AUTO_RESUME = TRUE. Sized for a dataset where the largest table (outpatient claims) is roughly 500,000 rows — well within what an XSMALL warehouse processes in seconds.
Database	One database (for example FINANCE_DB / PATIENT_DB) containing all schemas for this project.
Schema — raw / public	Holds the raw tables that the Kafka consumer writes into directly: PATEINT23, INPATIENT, OUTPATIENT, and FRAUD.
Schema — dwh (via dbt)	Holds the dimensional model built by dbt: all DIM_ tables, the FACT_TABLE, and the bridge tables.

5.3 Incremental Loading Into Snowflake

Incremental loading at the Snowflake landing layer is implemented at two levels.

At the Kafka consumer level, records are buffered in memory per topic and flushed to Snowflake in batches (for example, every 10,000 records) using the write_pandas utility from the Snowflake Python connector. This avoids the severe performance penalty of issuing one INSERT statement per row, while still keeping end-to-end latency low because batches flush frequently rather than waiting for a full daily file.

At the dbt level, incremental loading is implemented using dbt's incremental materialization, which is described in detail in the Incremental Loading section later in this document. In short, rather than rebuilding the entire dimensional model from scratch on every run, dbt models configured as incremental only process rows that are new since the last run, using a watermark column (such as a load timestamp or claim ID) to determine what counts as new.



6. Phase 4 — dbt Transformations

dbt (data build tool) is used to transform the raw Snowflake tables into a governed star schema. dbt was chosen because it allows every transformation to be written as version-controlled SQL, tested automatically, and documented through a built-in lineage graph — all properties that are difficult to achieve with ad-hoc SQL scripts or a black-box ETL tool.

6.1 Staging Models

Staging models are the first layer of dbt transformation. Each staging model selects directly from a Snowflake source table, performing only light cleaning — no business logic, no joins across sources, and no aggregation. The purpose of this layer is to create a clean, consistently named interface between the volatile raw tables and everything built on top of them.

Staging Model	Role
patient_stg	Selects and lightly cleans beneficiary demographic and chronic condition columns from the raw PATEINT23 table.
dim_Physician_stg	Unions the AttendingPhysician, OperatingPhysician, and OtherPhysician columns from both inpatient and outpatient claims into a single column of physician identifiers, the first step in building a role-playing physician dimension.
dim_location_stg	Extracts distinct State/County combinations from the patient data to prepare the location dimension.
dim_fraud_stg	Selects distinct Provider / PotentialFraud pairs from the raw fraud labels table.

dim_ClmDiagnosisCode_stg	Unpivots the ten ClmDiagnosisCode_1 through ClmDiagnosisCode_10 columns (plus the admission diagnosis code) from both inpatient and outpatient claims into a single long-format column of diagnosis codes.
dim_ClmProcedureCode_stg	Unpivots the six ClmProcedureCode_1 through ClmProcedureCode_6 columns from both claim sources into a single long-format column of procedure codes.
dim_DiagnosisGroup_code_stg	Extracts distinct DiagnosisGroupCode values from inpatient claims (this field only exists for inpatient stays).
dim_date_stg	Generates the full calendar date spine used to populate the date dimension, including year, quarter, month, day name, and weekend flag.

Engineering decision — why unpivot in staging:

The diagnosis and procedure codes arrive in the raw claims tables as repeating columns (ClmDiagnosisCode_1 through _10). This is a classic denormalization anti-pattern. Storing them this way in the dimensional model would make it impossible to write a simple WHERE clause to find all claims containing a particular diagnosis, would waste storage on a large number of NULL columns, and would not scale if a claim ever needed an eleventh diagnosis code. Unpivoting (using UNION ALL across each column position) at the staging layer converts this wide, denormalized shape into a long, normalized shape — one row per claim per diagnosis position — which is the shape required to populate both the diagnosis dimension and the bridge table correctly.

6.2 Dimension Models and the Star Schema

A star schema was chosen as the target dimensional model because the primary consumption pattern for this data is analytical slicing and aggregation in Power BI (for example, total reimbursement by provider, by month, by diagnosis category), not transactional updates. Star schemas are optimized for exactly this access pattern: a small number of large joins from a central fact table out to descriptive dimension tables, which Power BI's VertiPaq engine and DAX language are specifically optimized to handle.

The dimension tables built in this layer are:

- dim_Physician_dwh — a role-playing dimension of physician identifiers, surrogate-keyed, joined into the fact table three separate times (once each for AttendingPhysicianKey, OperatingPhysicianKey, and OtherPhysicianKey)
- dim_location_dwh — distinct State values with a surrogate key
- dim_fraud_dwh — distinct provider fraud labels with a surrogate key
- dim_gender_race_dwh — a junk dimension combining the low-cardinality Gender and Race fields into a single dimension to avoid two tiny, rarely-queried-alone dimension tables
- dim_ClmDiagnosisCode_dwh — the distinct list of diagnosis codes with a surrogate key, the target of the diagnosis bridge table

- `dim_ClmProcedureCode_dwh` — the distinct list of procedure codes with a surrogate key, the target of the procedure bridge table
- `dim_DiagnosisGroup_code_dwh` — distinct diagnosis-related group (DRG) codes, applicable only to inpatient claims
- `dim_date_dwh` — a standard date dimension, used as a role-playing dimension joined into the fact table multiple times (claim start date, claim end date, admission date, discharge date, date of birth, date of death)

Each dimension is built by selecting the distinct combination of attributes from the corresponding staging model and assigning a surrogate key using `ROW_NUMBER()`. Surrogate keys (rather than the natural business keys such as the raw physician ID or diagnosis code) are used as the join keys into the fact table for two reasons: they are guaranteed to be a stable, compact integer regardless of how the source system formats its natural keys, and they allow a dimension's natural key to change over time (a slowly changing dimension pattern) without requiring every fact row that references it to be rewritten.

6.3 Fact Table

The `fact_table` model is the centerpiece of the star schema. Its design required three explicit engineering decisions: the grain, the measures, and the foreign keys.

Grain

The grain of `fact_table` is defined as one row per claim. Every row corresponds to exactly one unique `ClaimID`, whether that claim originated from the inpatient or outpatient source table, distinguished by a `ClaimType` column ('IP' or 'OP'). This grain decision is the single most important design choice in the fact table, because it determines which columns are allowed to live in the fact table directly and which must be pushed out to a dimension or a bridge table. Any attribute that does not vary at the one-row-per-claim level — such as a patient's date of birth, which is constant across every claim that patient ever generates — does not belong in the fact table, because storing it there would mean repeating the same value across every claim row for that patient, which both wastes storage and breaks the one-fact-equals-one-business-event principle that makes a fact table queryable and aggregable in a predictable way.

Measures

The fact table's measures are the numeric columns that are additive (or at least aggregable) across the grain: `InscClaimAmtReimbursed` and `DeductibleAmtPaid`, which are specific to the individual claim, plus a derived `LengthOfStay` measure, calculated as the difference in days between `DischargeDt` and `AdmissionDt` (and only populated for inpatient claims, since outpatient claims have no admission/discharge period).

Foreign Keys

The fact table holds a foreign key to every dimension described above. The physician and date dimensions are joined multiple times under different aliases (role-playing dimensions), since a

single claim references several different physicians and several different dates simultaneously, but all of those references point back to the same underlying dimension table rather than requiring separate physician or date dimensions for each role.

Business Logic Embedded in the Fact Table

- ClaimType differentiates inpatient from outpatient claims within a single unified fact table, avoiding the need for two parallel fact tables and two parallel sets of Power BI measures.
- LengthOfStay is calculated once during the dbt build rather than being recalculated in every downstream Power BI report, ensuring consistency across all dashboards that use this metric.
- NULL admission and discharge dates for outpatient claims are handled explicitly through LEFT JOINS against the date dimension, so that outpatient claims do not get silently dropped from the fact table by an INNER JOIN.

6.4 Bridge Tables

Two bridge tables were built: bridge_claim_diagnosis and bridge_claim_procedure.

Why Diagnosis and Procedure Codes Cannot Live Directly in the Fact Table

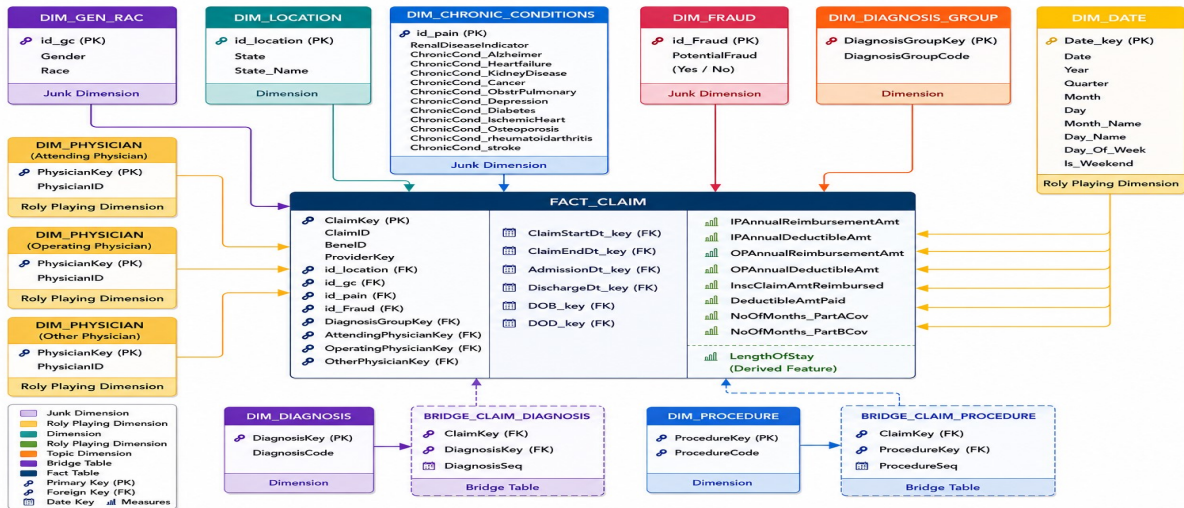
A single claim can have between zero and ten diagnosis codes and between zero and six procedure codes. This is a many-to-many relationship: one claim can reference many diagnosis codes, and one diagnosis code (for example, a common code such as diabetes) appears across many different claims. A fact table grain of one-row-per-claim is fundamentally incompatible with storing a variable number of diagnosis codes per row — doing so would either require repeating the claim's measures across multiple rows (one per diagnosis), which violates the fact table's defined grain and would cause every SUM() of InscClaimAmtReimbursed to be wildly overstated, or it would require going back to the original wide, repeating-column format, which is unqueryable at scale.

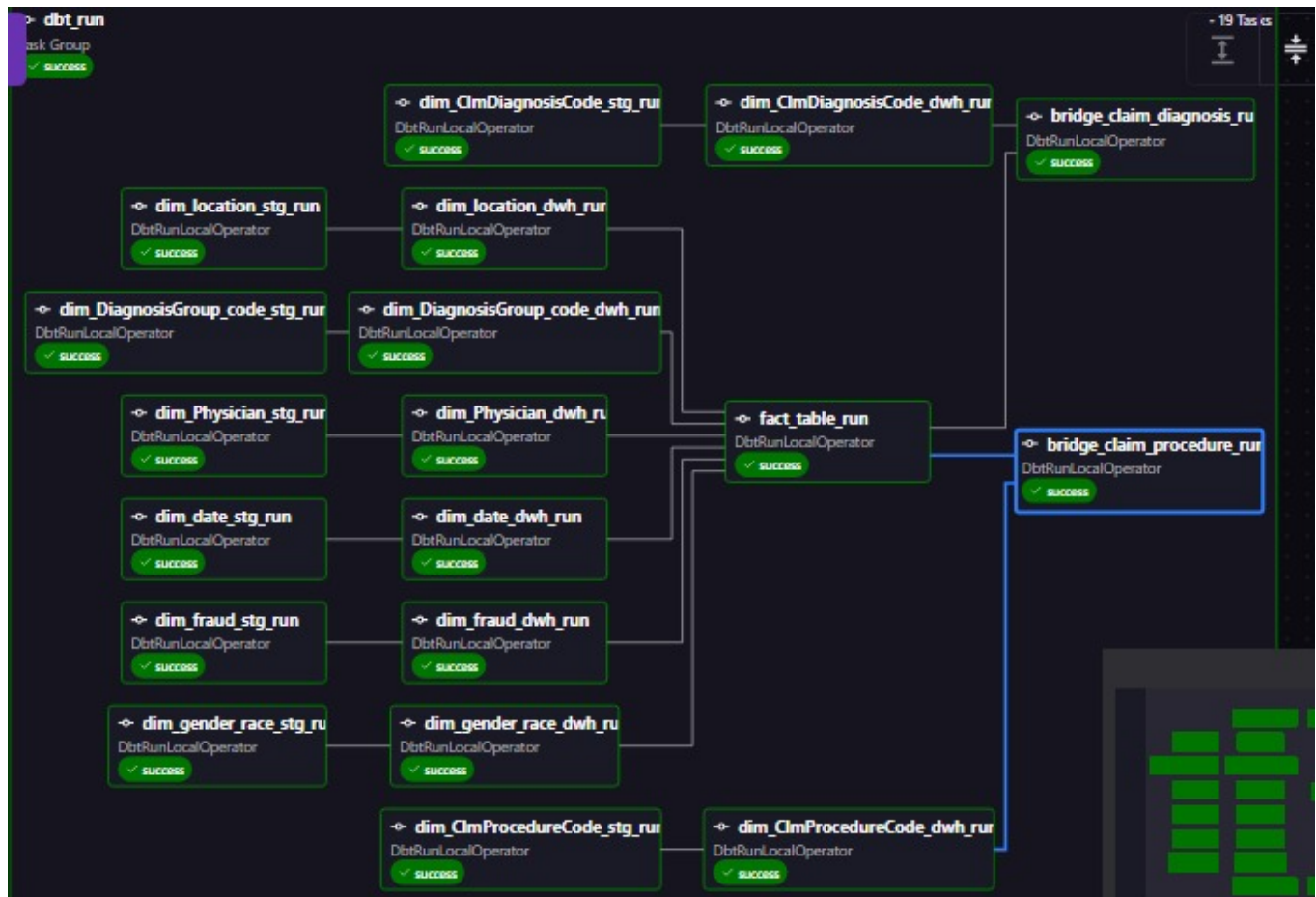
The standard data warehousing solution to a many-to-many relationship between a fact and a dimension is a bridge table (also called an associative or factless table). The bridge table holds one row per claim-diagnosis pairing, referencing the fact table's surrogate key and the dimension's surrogate key, plus a sequence number indicating the original position of that diagnosis on the claim (0 for the admission diagnosis, 1 through 10 for the listed claim diagnoses).

Structure

bridge_claim_diagnosis	bridge_claim_procedure
ClaimKey (FK -> fact_table)	ClaimKey (FK -> fact_table)
DiagnosisKey (FK -> dim_ClaimDiagnosisCode_dwh)	ProcedureKey (FK -> dim_ClaimProcedureCode_dwh)
DiagnosisSeq (0 for admission, 1-10 for claim diagnoses)	ProcedureSeq (1-6)

With this structure, a query asking 'how many claims include diagnosis code X' becomes a simple filtered join on bridge_claim_diagnosis, rather than a sprawling OR condition across ten separate columns. Power BI relationships are configured against the bridge tables using a many-to-many relationship type, which Power BI's composite model engine supports natively, allowing diagnosis- and procedure-level slicing to work correctly without double-counting claim-level measures.





7. Phase 5 — Databricks Medallion Architecture(ELT)

In parallel to the dbt-built star schema in Snowflake, a second processing path was built in Databricks using the Medallion Architecture pattern (Bronze, Silver, Gold). This path exists to support heavier, more flexible data engineering and advanced analytics workloads — such as larger-scale data quality validation, feature engineering for potential machine learning fraud scoring, and processing logic that is more naturally expressed in PySpark than in SQL — while the dbt/Snowflake star schema remains the system of record for standard BI reporting.

7.1 Bronze Layer

The Bronze layer is the raw, unmodified landing zone. Data is ingested from source (in this project, from Snowflake or directly from the source files) and written into Delta Lake tables exactly as received, with no cleaning, no type casting, and no business logic applied. The purpose of Bronze is to preserve a complete, auditable copy of the data exactly as it arrived, so

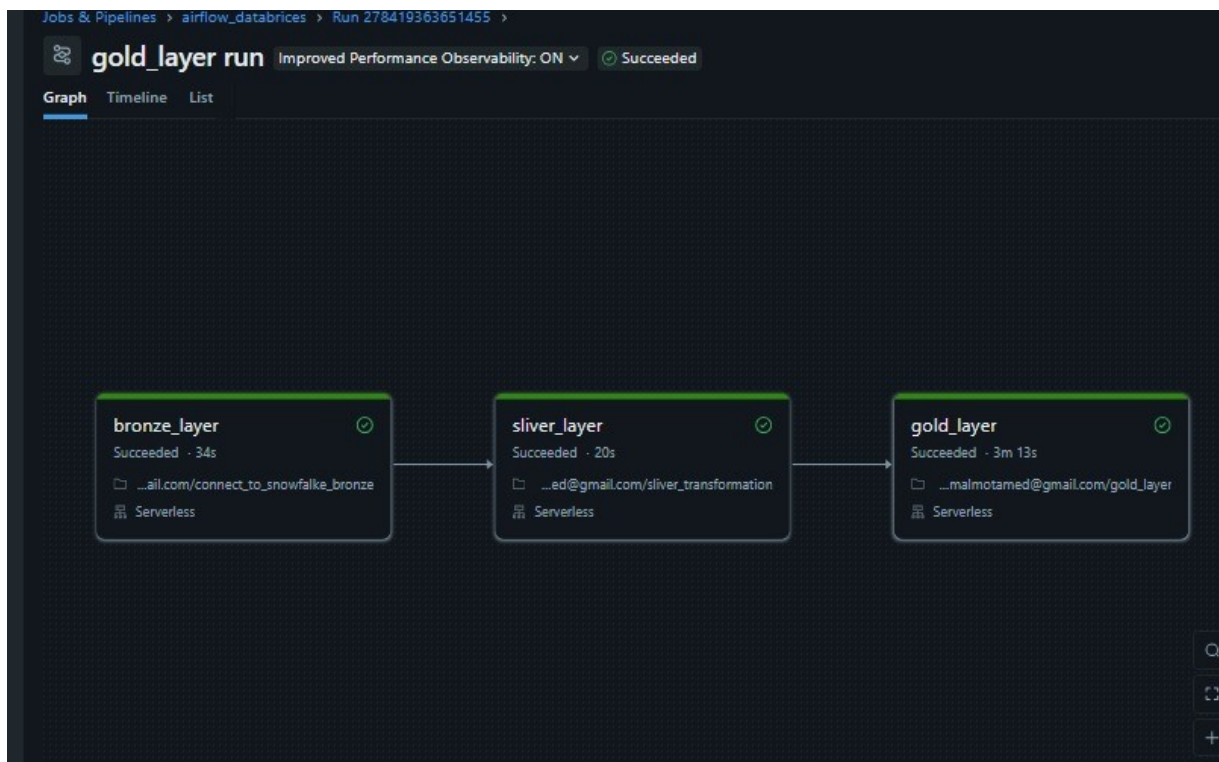
that if a downstream transformation bug is later discovered, the pipeline can be replayed from the original source data rather than from an already-modified version.

7.2 Silver Layer

Load data from bronze layer and it was data warehouse for all dim and fact tables and raw data

7.3 Gold Layer

Made all transformation in this layer



8. Phase 6 — Incremental Loading

Incremental loading is the strategy that allows this platform to process only new or changed data on each run, rather than reprocessing the entire historical dataset every time the pipeline executes. This is essential for a platform that is expected to receive new claims continuously, since reprocessing half a million or more rows on every run would be both slow and unnecessarily expensive.

8.1 Append Strategy

The simplest incremental pattern used in this platform is the append strategy, used at the Kafka-consumer-to-Snowflake boundary. Since claims are, by nature, immutable once submitted (a historical claim record does not get edited after the fact in this dataset), new records can simply be appended to the raw Snowflake tables as they arrive, with no need to check for or merge against existing rows.

8.2 Deduplication

Despite claims being logically immutable, deduplication logic is still required because Kafka's at-least-once delivery guarantee means a message could, in rare failure scenarios (such as a consumer crash between writing to Snowflake and committing its offset), be delivered and processed more than once. To guard against this, the dbt staging layer applies deduplication using a `QUALIFY ROW_NUMBER() OVER (PARTITION BY ClaimID ORDER BY load_timestamp DESC) = 1` pattern (or an equivalent `DISTINCT`-based approach), ensuring that even if a duplicate message slips through Kafka, only one copy of that claim ever reaches the dimensional model.

8.3 Merge / Upsert Concepts

Where data can genuinely change after first being loaded (for example, the fraud provider data, where a provider's PotentialFraud label could be updated after further investigation), a merge (also called an upsert) pattern is used instead of a simple append. A `MERGE` statement compares incoming records against existing records on a business key (such as Provider) and either updates the existing row if the key already exists or inserts a new row if it does not. dbt's incremental materialization supports this directly through the merge incremental strategy, which generates the appropriate Snowflake `MERGE INTO` statement automatically based on a configured `unique_key`.

9. Phase 7 — Orchestration

Databricks Workflows is used to orchestrate the scheduled execution of the pipeline's processing tasks, replacing the role that a tool such as Apache Airflow or SQL Server Agent would play in a more traditional architecture.

9.1 Workflow Design

Two primary tasks were defined within the Databricks Workflow:

- `incremental_load` — reads newly arrived data and writes it into the Bronze and Silver layers, applying the cleaning, standardization, and deduplication logic described in the Incremental Loading section
- `gold_layer` — runs after `incremental_load` completes successfully, rebuilding or incrementally updating the curated Gold tables from the latest Silver data

The screenshot displays the Databricks Jobs & Pipelines interface for a workflow named 'gold_layer run'. The workflow is shown in a graph view, indicating a successful execution. The first task, 'incrmental_load', completed in 1 minute and 0 seconds. The second task, 'gold_layer', completed in 3 minutes and 38 seconds. The 'Details' panel on the right provides specific information about the job run, including the Job ID (691842041089885), Job run ID (695876305967019), Task run ID (1026432418004020), and the user 'amalmotamed@gmail.com'. The status is 'Succeeded', and the performance optimization is 'Enabled'.

9.2.Dependencies and Scheduling

The `gold_layer` task is configured with an explicit dependency on the successful completion of `incrmental_load`, meaning Databricks Workflows will not attempt to build Gold-layer tables from Silver data that has not yet been refreshed. This dependency-based execution model — rather than simply scheduling both tasks to run at fixed times independently — guarantees data consistency between layers even if one task runs longer than expected on a given day.

The workflow itself is scheduled to run on a recurring basis aligned with the expected arrival pattern of new claims data, ensuring that the Gold layer (and downstream Power BI reports built against it) reflects newly arrived data within a predictable, bounded delay.

10. ssis,ssas, dash board Overview

10.2. Database Schema Design

Dimension Tables

Tables were designed using `IDENTITY(1,1)` for Surrogate Keys, ensuring complete isolation from source systems:

```
-- 1. Diagnosis dictionary
CREATE TABLE DIM_DIAGNOSIS (
    DiagnosisKey INT IDENTITY(1,1) PRIMARY KEY,
    DiagnosisCode NVARCHAR(50) NOT NULL UNIQUE
);
```

```

-- 2. Procedure dictionary
CREATE TABLE DIM_PROCEDURE (
    ProcedureKey INT IDENTITY(1,1) PRIMARY KEY,
    ProcedureCode NVARCHAR(50) NOT NULL UNIQUE
);

-- 3. Demographics dimension (Gender & Race)
CREATE TABLE DIM_GEN_RAC (
    id_gc INT IDENTITY(1,1) PRIMARY KEY,
    Gender NVARCHAR(20),
    Race NVARCHAR(50)
);

-- 4. Geographic location dimension
CREATE TABLE DIM_LOCATION (
    id_location INT IDENTITY(1,1) PRIMARY KEY,
    State NVARCHAR(10)
);

-- 5. Fraud indicator dimension
CREATE TABLE DIM_FRAUD (
    id_Fraud INT IDENTITY(1,1) PRIMARY KEY,
    PotentialFraud NVARCHAR(50)
);

-- 6. Physicians dimension
CREATE TABLE DIM_PHYSICIAN (
    PhysicianKey INT IDENTITY(1,1) PRIMARY KEY,
    AttendingPhysician NVARCHAR(50) NULL,
    OperatingPhysician NVARCHAR(50) NULL,
    OtherPhysician NVARCHAR(50) NULL
);

-- 7. Patient dimension (supports the wide build and chronic-condition flags)
CREATE TABLE DIM_PATIENT (
    PatientKey INT IDENTITY(1,1) PRIMARY KEY,
    BeneID NVARCHAR(50) NULL, -- Business Key for linkage
    RenalDiseaseIndicator NVARCHAR(10) NULL,
    ChronicCond_Alzheimer NVARCHAR(10) NULL,
    ChronicCond_Heartfailure NVARCHAR(10) NULL,
    ChronicCond_KidneyDisease NVARCHAR(10) NULL,
    ChronicCond_Cancer NVARCHAR(10) NULL,
    ChronicCond_ObstrPulmonary NVARCHAR(10) NULL,
    ChronicCond_Depression NVARCHAR(10) NULL,
    ChronicCond_Diabetes NVARCHAR(10) NULL,
    ChronicCond_IschemicHeart NVARCHAR(10) NULL,
    ChronicCond_Osteoporosis NVARCHAR(10) NULL,

```

```

        ChronicCond_Rheumatoidarthritis NVARCHAR(10) NULL,
        ChronicCond_stroke NVARCHAR(10) NULL
    );

-- 8. Shared Date Dimension
CREATE TABLE DIM_DATE (
    Date_key INT PRIMARY KEY,
    [Date] DATE,
    [Year] INT,
    [Quarter] INT,
    [Month] INT,
    [Day] INT,
    Month_Name NVARCHAR(50),
    Day_Name NVARCHAR(50),
    Day_Of_Week INT,
    Is_Weekend BIT
);

```

Bridge Tables (Many-to-Many Relationships)

To resolve the issue of a single claim having multiple diagnoses and multiple procedures without duplicating columns in the Fact table:

```

CREATE TABLE BRIDGE_CLAIM_DIAGNOSIS (
    ClaimID NVARCHAR(50) NOT NULL,
    DiagnosisKey INT NOT NULL
);

CREATE TABLE BRIDGE_CLAIM_PROCEDURE (
    ClaimID NVARCHAR(50) NOT NULL,
    ProcedureKey INT NOT NULL
);

```

Main Fact Table (FACT_CLAIMS)

Contains only the surrogate keys linking to the dimensions, plus the numeric measures used for calculations:

```

CREATE TABLE FACT_CLAIMS (
    ClaimID NVARCHAR(50) NOT NULL PRIMARY KEY,
    BeneID NVARCHAR(50) NOT NULL,
    Provider NVARCHAR(50) NULL,
    Claim_Type NVARCHAR(50) NULL, -- Degenerate Dimension for fast filtering (In/Out)

    -- Surrogate Keys
    PatientKey INT NULL,
    PhysicianKey INT NULL,
    id_gc INT NULL,
    id_location INT NULL,

```

```
id_Fraud INT NULL,

-- Role-Playing Date Keys
ClaimStart_DateKey INT NULL,
ClaimEnd_DateKey INT NULL,
Admission_DateKey INT NULL,
Discharge_DateKey INT NULL,
Birth_DateKey INT NULL,
Death_DateKey INT NULL,

-- Measures
InscClaimAmtReimbursed DECIMAL(18,2) NULL,
DeductibleAmtPaid DECIMAL(18,2) NULL,
NoOfMonths_PartACov INT NULL,
NoOfMonths_PartBCov INT NULL,
IPAnnualReimbursementAmt DECIMAL(18,2) NULL,
IPAnnualDeductibleAmt DECIMAL(18,2) NULL,
OPAnnualReimbursementAmt DECIMAL(18,2) NULL,
OPAnnualDeductibleAmt DECIMAL(18,2) NULL
);
```

10.3. ETL / SSIS Architecture

Overall Control Flow

Packages were organized inside a Sequence Container to ensure all dimensions are populated first, in the correct parallel/sequential order, preceded by a full data-cleanup step, and followed at the end by the Fact package.

Pre-Execution Cleanup Step

An Execute SQL Task is run to clear the tables and reset counters before each run, following this structural order to avoid any relational issues:

```
TRUNCATE TABLE FACT_CLAIMS;
TRUNCATE TABLE BRIDGE_CLAIM_DIAGNOSIS;
TRUNCATE TABLE BRIDGE_CLAIM_PROCEDURE;
TRUNCATE TABLE DIM_DIAGNOSIS;
TRUNCATE TABLE DIM_PROCEDURE;
TRUNCATE TABLE DIM_GEN_RAC;
TRUNCATE TABLE DIM_LOCATION;
TRUNCATE TABLE DIM_FRAUD;
TRUNCATE TABLE DIM_PHYSICIAN;
TRUNCATE TABLE DIM_PATIENT;
```

Bridge Tables Data Flow

We faced the challenge of unpivoting 10 diagnosis columns and 6 procedure columns horizontally, which was solved with the following scenario implemented in the packages:

1. OLE DB Source: a custom query protects the pipeline and standardizes data types, handling nulls and whitespace using:

```
SELECT [ClaimID], ISNULL(TRIM(CAST([ClmDiagnosisCode_1] AS NVARCHAR(50))), '') AS
[ClmDiagnosisCode_1], ...
```

2. Unpivot Transformation: converts the 10 wide columns into a single unified column called Diagnosis_Column or Procedure_Column.
3. Conditional Split: excludes empty records produced by the transformation, shrinking the dataset down to useful rows only, via the condition: Diagnosis_Column != "".
4. Lookup Transformation: matches the resulting codes against the corresponding dimension table (e.g. DIM_DIAGNOSIS) to retrieve the matching Surrogate Key.
5. OLE DB Destination: loads the final output, consisting of (ClaimID + Key), directly into the Bridge table.

Fact Table Data Flow

This consists of a sequential pipeline that starts by reading the OBT from STG, then passes through 11 consecutive Lookup stages:

- The first 5 stages: link the standard dimensions and retrieve their surrogate keys (PatientKey, PhysicianKey, etc.).
- The remaining 6 stages: apply the Role-Playing Dimension tactic against the unified DIM_DATE table, linking every claim date (start, end, admission, discharge, birth, death) to the appropriate DateKey, with the Ignore Failure option enabled for cases that may contain null values, such as death or admission dates.
- Final destination: routes and loads all keys and financial measures into the FACT_CLAIMS table.

10.4. Advanced Engineering Tactics Applied in the Project

Tactic	Description
Many-to-Many Resolution	Resolving the many-to-many relationship between medical claims, diagnoses, and procedures using Bridge Tables linked by ClaimID, which protects the Fact table from bloat and duplication and keeps DAX calculations performant.
Role-Playing Dimension	Using a single date table to serve 6 different time-based requirements within one fact table.
Degenerate Dimension	Keeping Claim_Type inside the Fact table to boost the performance of core filtering on report dashboards.
Data Cleaning & Trimming	Pre-processing whitespace and blanks to protect the SSIS components from unexpected failures (DTS_E_PROCESSINPUTFAILED).

10.5. Detailed ETL Steps in SSIS

The ETL (Extract, Transform, Load) steps inside SSIS are the backbone of any Data Warehouse project. SSIS splits the work into two main levels: Control Flow (manages the order in which packages and tasks run), and inside it, Data Flow (the production line through which each row actually travels from source to destination).

Below is a detailed, step-by-step engineering walkthrough from the very beginning all the way to the final load:

I. Extraction Phase

In this phase, the goal is to connect to data sources (whether Staging tables, Excel files, or flat files) and pull the data into the Pipeline (SSIS's in-memory buffer).

Creating Connection Managers

Before any extraction, SSIS needs to know where to go. Right-click in the Connections area at the bottom and create a connection to the source (e.g. an OLE DB Connection to the STG database) and a connection to the destination (an OLE DB Connection to the DWH).

Adding the OLE DB Source Component

Drag this component into the Data Flow Task.

Choosing the Data Access Mode

- Table or View: used when reading the raw table as-is (the simple option).
- SQL Command (the better engineering choice): write a custom SELECT query to pull only the needed columns and perform initial processing (such as CAST to standardize data types, or TRIM to clean whitespace) to ensure the downstream steps don't fail.

II. Transform & Cleaning Phase

Once the data leaves the source, it becomes Data Streams flowing along blue arrows. Here we begin applying transformation, cleaning, and routing operations:

Structural Transformations

- Unpivot: converts wide columns (such as diagnosis columns 1 through 10) into long-form rows to prepare the data for the Bridge tables.
- Derived Column: a powerful tool for creating new columns based on expressions. Used here to create flags (such as converting an admission date into the label Inpatient or Outpatient).
- Data Conversion: changes a column's data type (e.g. converting from string [DT_STR] to Unicode string [DT_WSTR]) when source and destination types don't match.

Cleaning & Filtering

- Conditional Split: acts like a "traffic officer." Based on certain conditions (e.g. Code != "") it splits the data into separate outputs — rows matching the condition continue, while non-matching rows are routed to a discard or logging path.
- Sort: orders the data and removes duplicates (Remove rows with duplicate sort values) when populating a Dimension table and only unique values are needed.

Business Logic & Lookups

- Lookup Transformation: the most important tool in the DWH. It takes a natural/business key (such as BenelD or State) and looks it up in the corresponding dimension table inside the DWH to retrieve the matching Surrogate Key.
- Slowly Changing Dimension (SCD) Wizard: a built-in tool used with dimension tables (such as Patients) to manage history (SCD Type 2) — opening a new row when the patient's data changes and closing the old row with dates and an IsCurrent flag.

III. Load Phase

After the data has been cleaned, split, and had its keys resolved, it reaches its final stop: the actual write to disk into the Data Warehouse tables.

Adding the OLE DB Destination Component

Drag it to the end of the Data Flow and connect the blue arrow (carrying the clean data and keys) to it.

Choosing the Destination Table

Open the component and select the target DWH table (whether a DIM, BRIDGE, or FACT table).

Choosing the Data Access Mode for Loading

- Always prefer Table or view - fast load: this enables Bulk Insert, allowing SSIS to load thousands of rows in a fraction of a second instead of row-by-row writes.
- Enable Keep Nulls if you want empty fields to be written as explicit NULLs in the database instead of falling back to default values.

Final Column Mapping

Manually map the pipeline columns (on the right) to the actual destination table columns in the database (on the left), then click OK.

IV. Control Flow Orchestration

After finishing the Data Flows for each table, step back out to the main Control Flow canvas to organize the overall workflow:

- Execute SQL Task (Cleanup): placed as the very first step, running TRUNCATE statements to clear out old data from the tables before the new load.
- Sequence Container (Grouping): groups all the Dimension Data Flow Tasks together so they can run in parallel, improving overall run time.
- Precedence Constraints (Green Arrows): connects the Container to the Fact Data Flow Task with a green arrow, meaning: "SSIS must not touch or start loading the Fact table until the Container has completed successfully (turned green) and all Dimensions and Bridges have been populated" — preventing any Foreign Key violations.

10.6. Appendix - SSIS Workspace Screenshots

This section contains screenshots of the SSIS Control Flow and Data Flow components for each workspace: STG, ODS, and DWH.

STG (Staging) Workspace

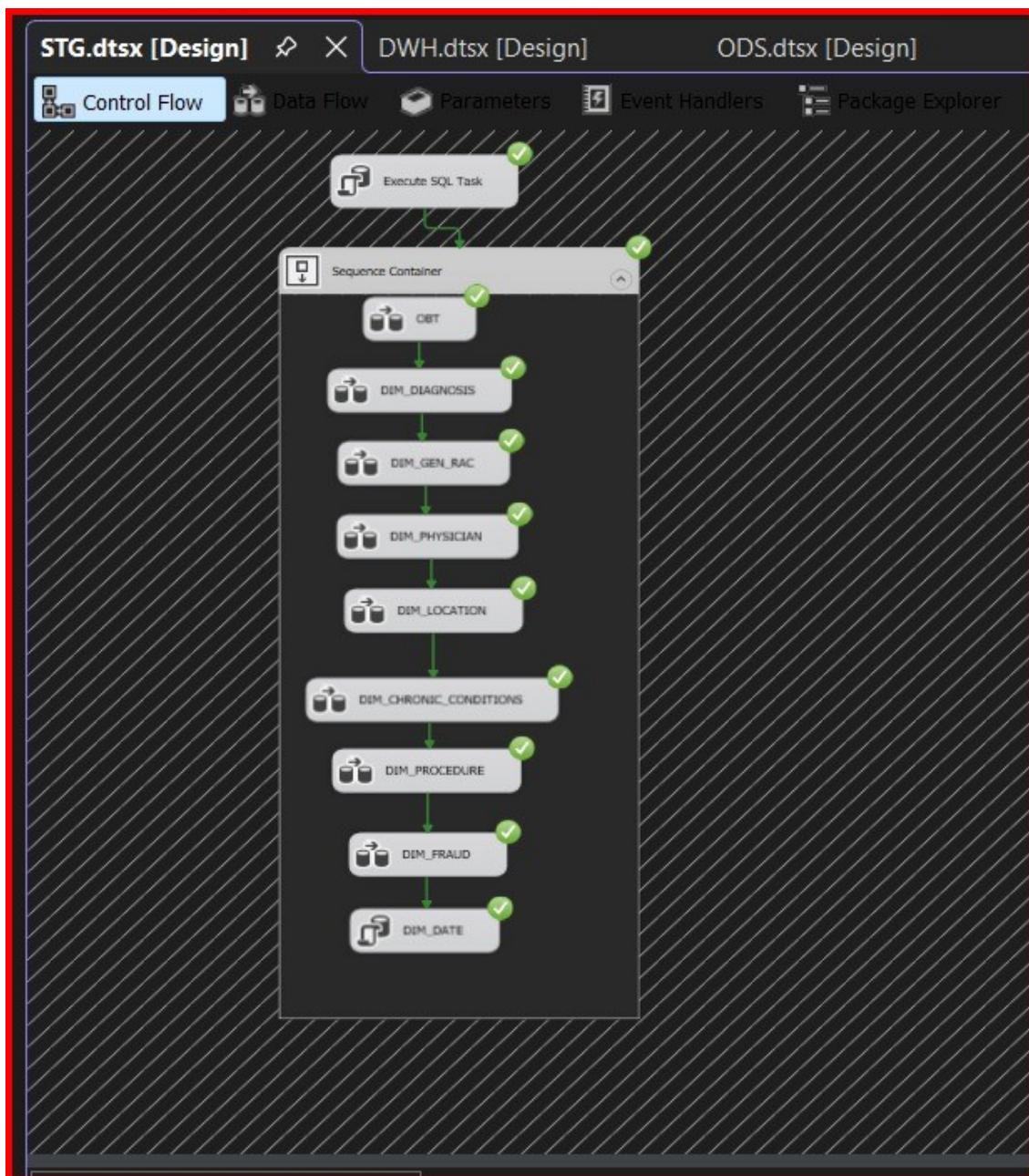


Figure 1: STG.dtsx Control Flow - Sequence Container loading OBT and all dimension tables (DIM_DIAGNOSIS, DIM_GEN_RAC, DIM_PHYSICIAN, DIM_LOCATION, DIM_CHRONIC_CONDITIONS, DIM_PROCEDURE, DIM_FRAUD, DIM_DATE).

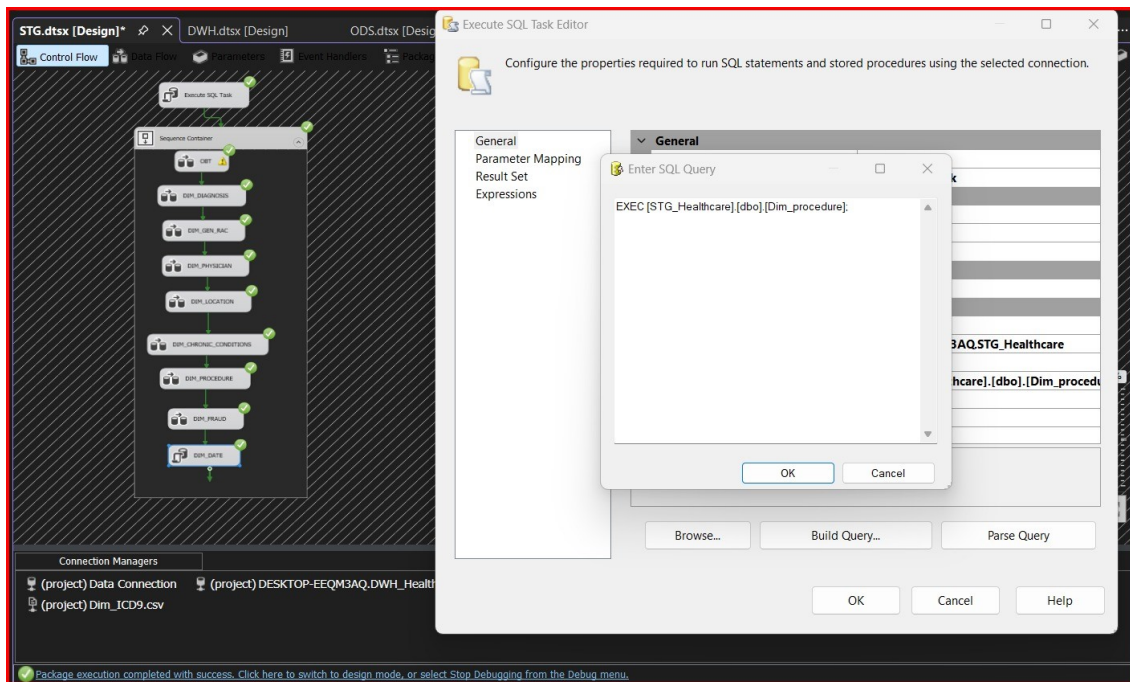


Figure 2: Execute SQL Task Editor - EXEC statement calling [STG_Healthcare].[dbo].[Dim_procedure], shown alongside the Control Flow.



Figure 3: STG.dtsx Data Flow for the OBT package - OLE DB Source -> Data Conversion -> OLE DB Destination (558,211 rows).

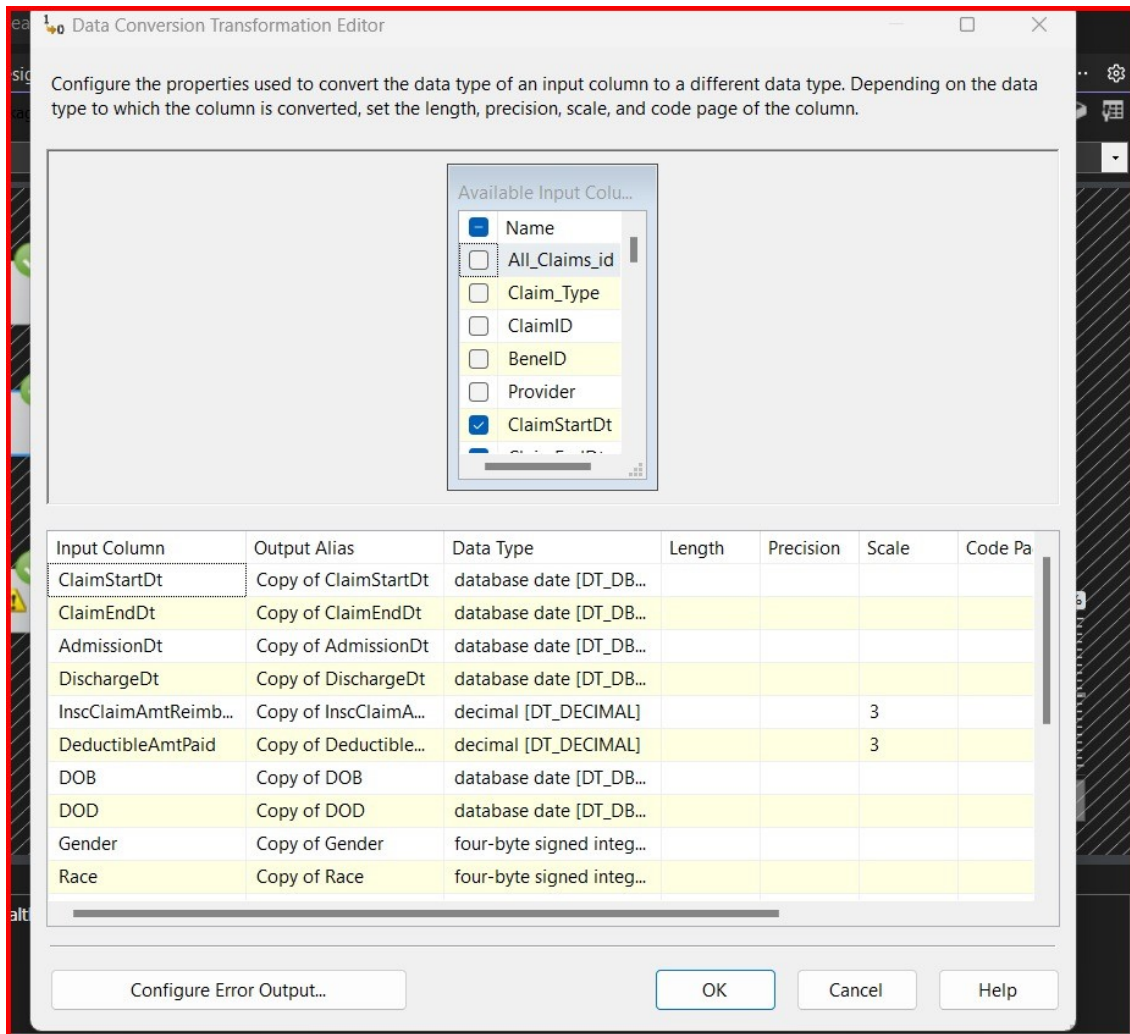


Figure 4: Data Conversion Transformation Editor - converting date and decimal columns (ClaimStartDt, ClaimEndDt, AdmissionDt, DischargeDt, InscClaimAmtReimbursed, DeductibleAmtPaid, DOB, DOD, Gender, Race) to their target data types.



Figure 5: Simple OLE DB Source -> OLE DB Destination flow loading 558,211 rows into a dimension/staging table.

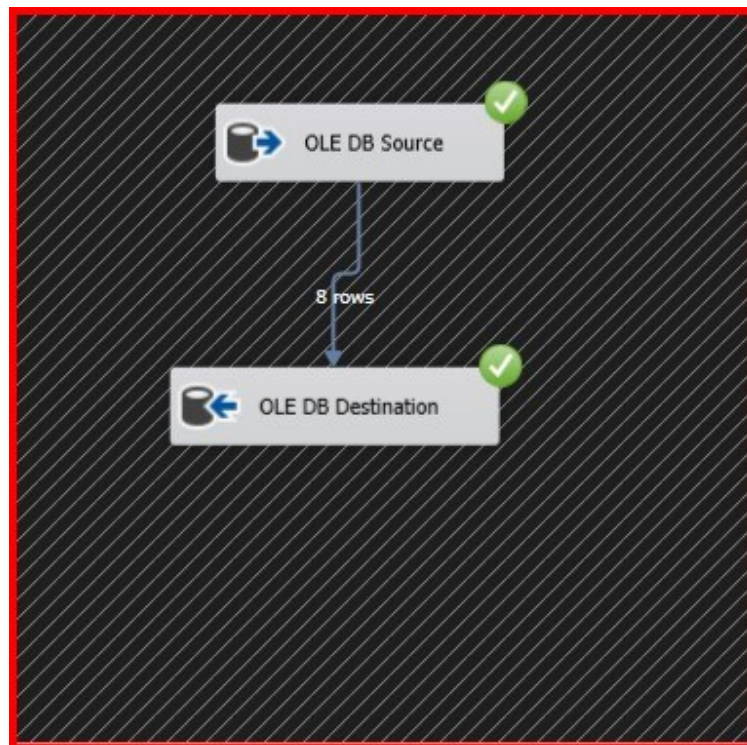


Figure 6: Simple OLE DB Source -> OLE DB Destination flow loading a small reference table (8 rows).

ODS (Operational Data Store) Workspace

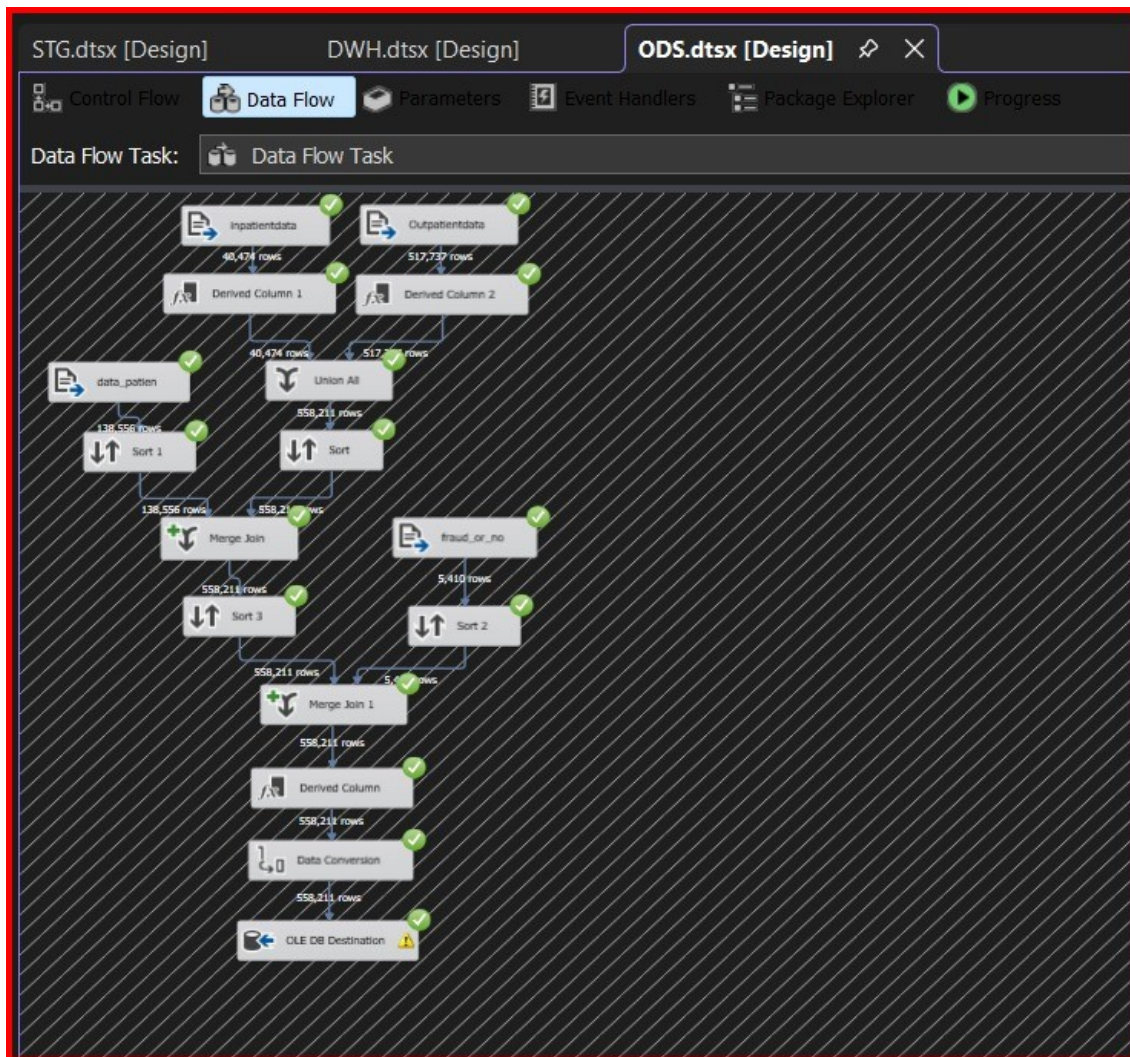


Figure 7: ODS.dtsx Data Flow - Inpatientdata and Outpatientdata sources merged via Derived Column, Union All, Sort, and Merge Join steps (with fraud_or_no), followed by Derived Column, Data Conversion, and OLE DB Destination (558,211 rows).

DWH (Data Warehouse) Workspace

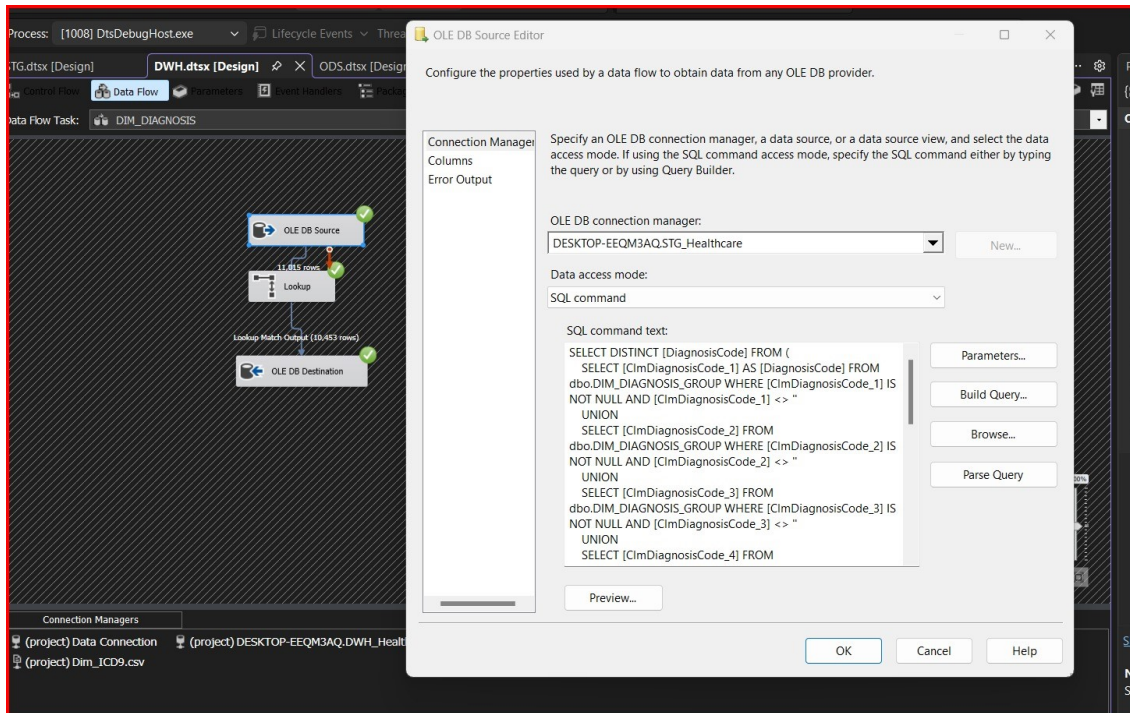


Figure 8: DWH.dtsx Data Flow for DIM_DIAGNOSIS - OLE DB Source Editor with the SQL command selecting distinct diagnosis codes (UNION across ClmDiagnosisCode_1..n) from the STG_Healthcare database.

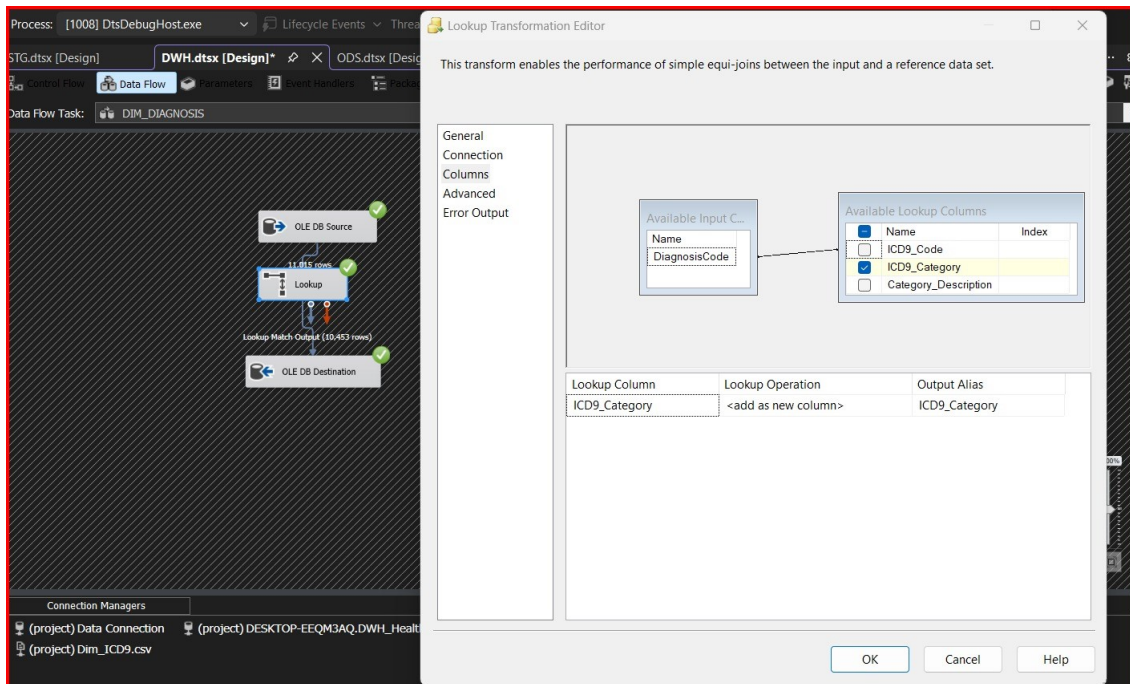


Figure 9: Lookup Transformation Editor for DIM_DIAGNOSIS - matching DiagnosisCode against the ICD9 reference list to pull ICD9_Category.

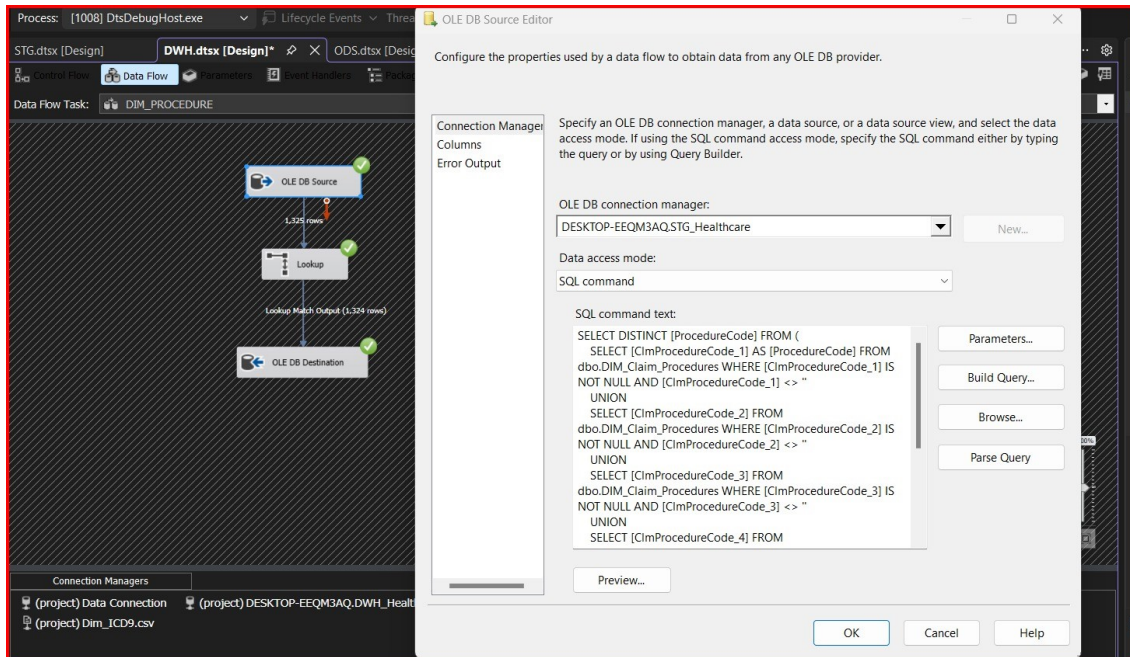


Figure 10: OLE DB Source Editor for DIM_PROCEDURE - SQL command selecting distinct procedure codes (UNION across ClmProcedureCode_1..n) from dbo.DIM_Claim_Procedures.

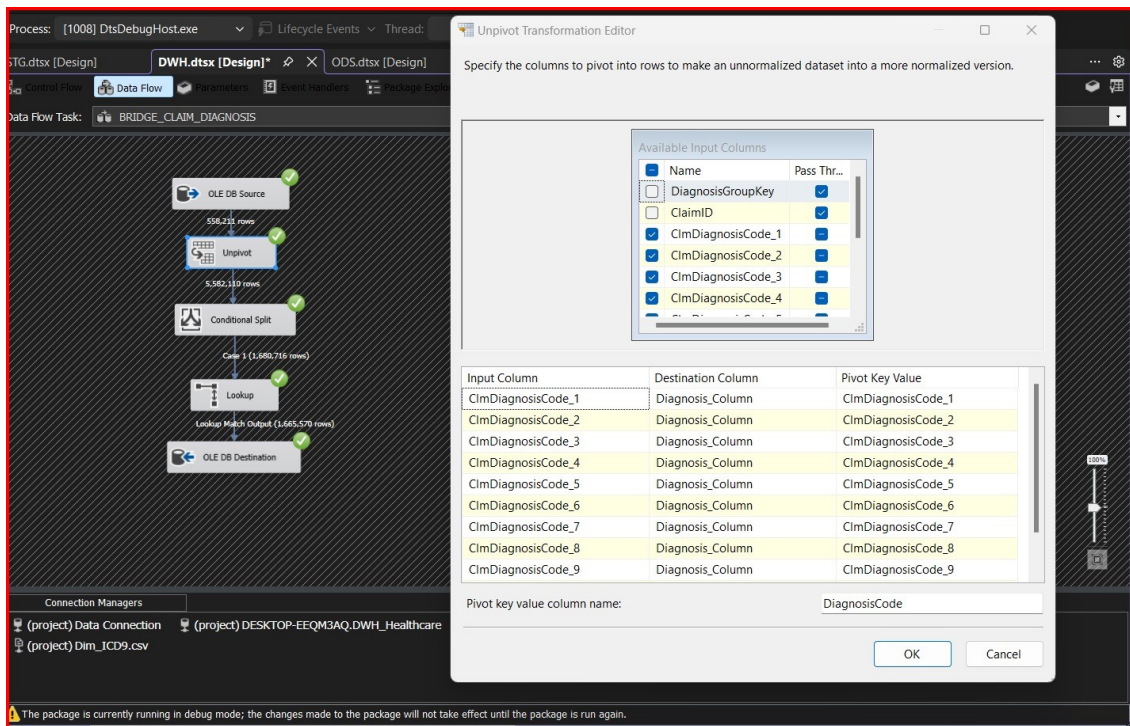


Figure 11: Unpivot Transformation Editor for BRIDGE_CLAIM_DIAGNOSIS - converting ClmDiagnosisCode_1 through _9+ into a single Diagnosis_Column with pivot key value column DiagnosisCode (558,211 -> 5,582,110 rows).

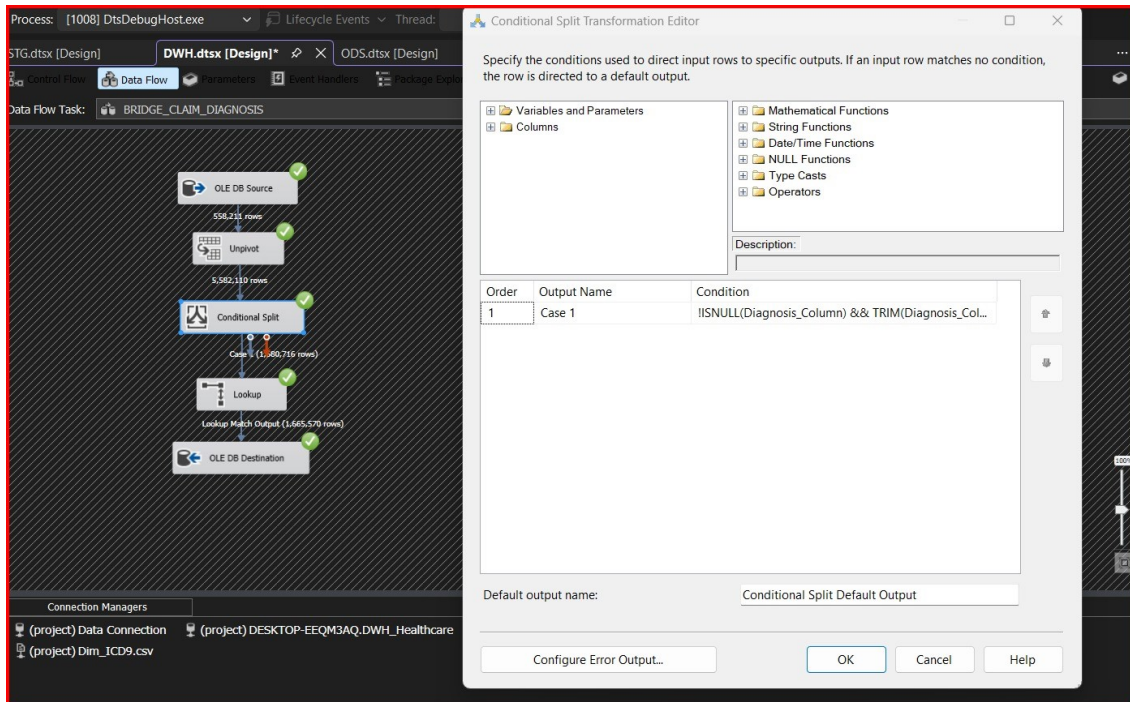


Figure 12: Conditional Split Transformation Editor for BRIDGE_CLAIM_DIAGNOSIS - filtering with !!ISNULL(Diagnosis_Column) && TRIM(Diagnosis_Column) condition (Case 1: 1,680,716 rows).

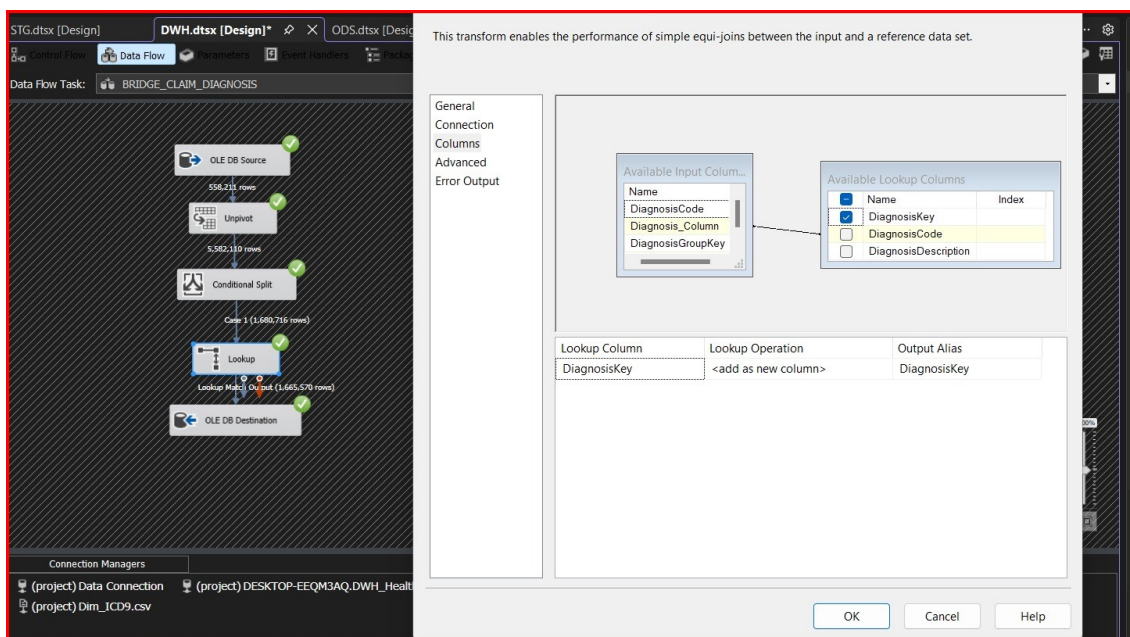


Figure 13: Lookup Transformation Editor for BRIDGE_CLAIM_DIAGNOSIS - resolving DiagnosisCode to DiagnosisKey before loading into the bridge table.

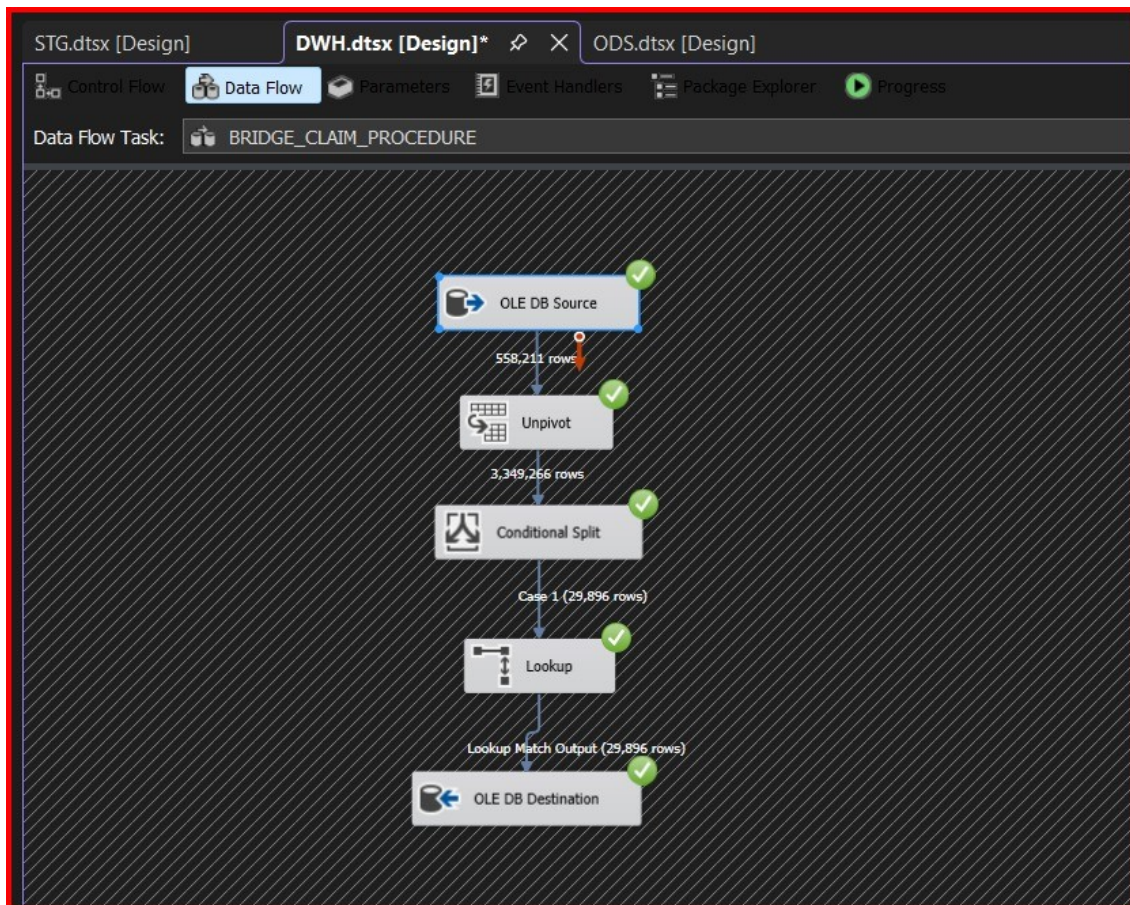


Figure 14: DWH.dtsx Data Flow for BRIDGE_CLAIM_PROCEDURE - OLE DB Source -> Unpivot -> Conditional Split -> Lookup -> OLE DB Destination (558,211 -> 3,349,266 -> 29,896 rows).

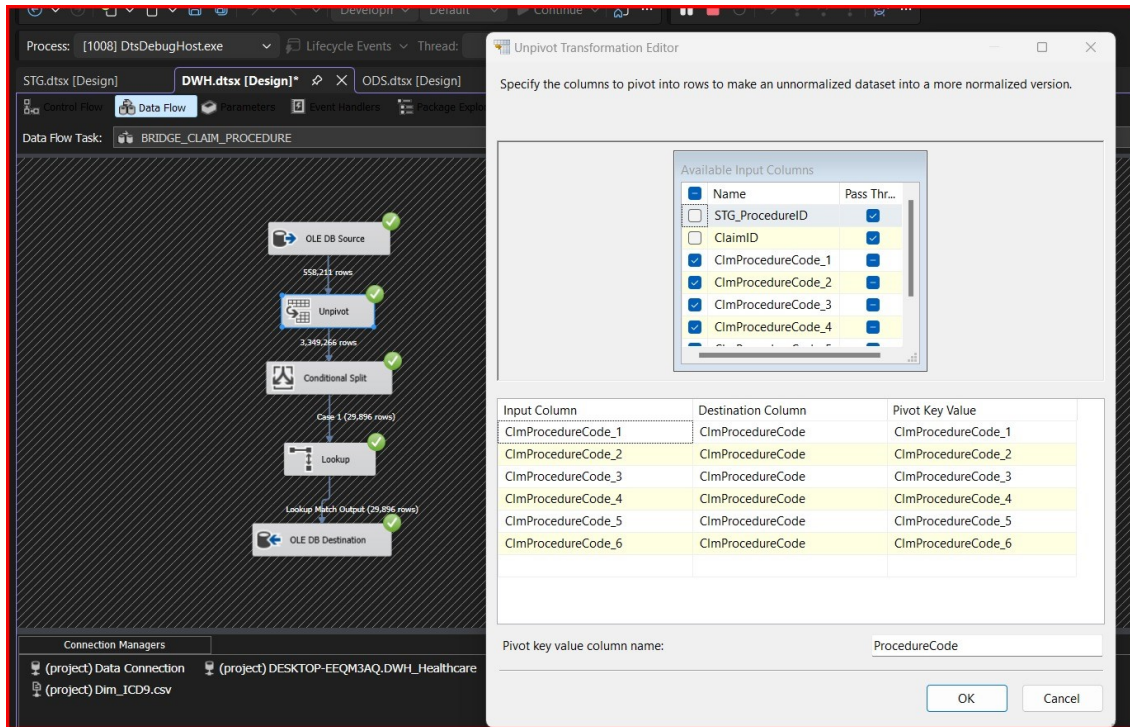


Figure 15: Unpivot Transformation Editor for BRIDGE_CLAIM_PROCEDURE - converting ClimProcedureCode_1 through _6 into a single ClimProcedureCode column with pivot key value column ProcedureCode.

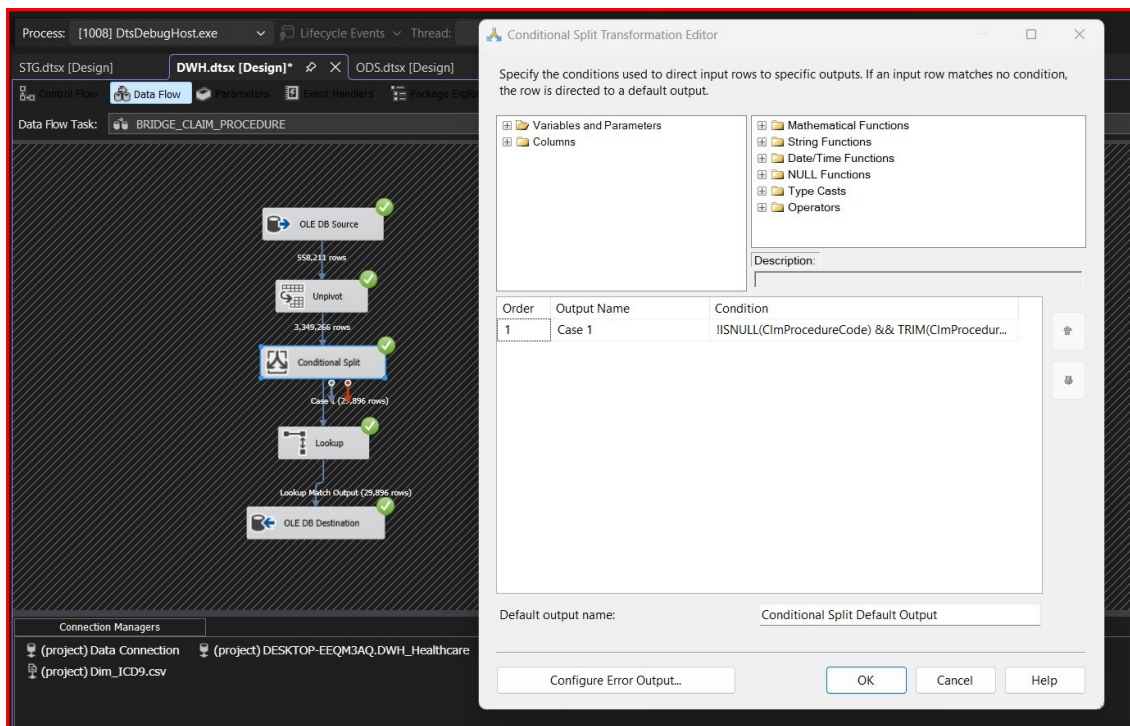


Figure 16: Conditional Split Transformation Editor for BRIDGE_CLAIM_PROCEDURE - filtering with !ISNULL(ClimProcedureCode) && TRIM(ClimProcedureCode) condition (Case 1: 29,896 rows).

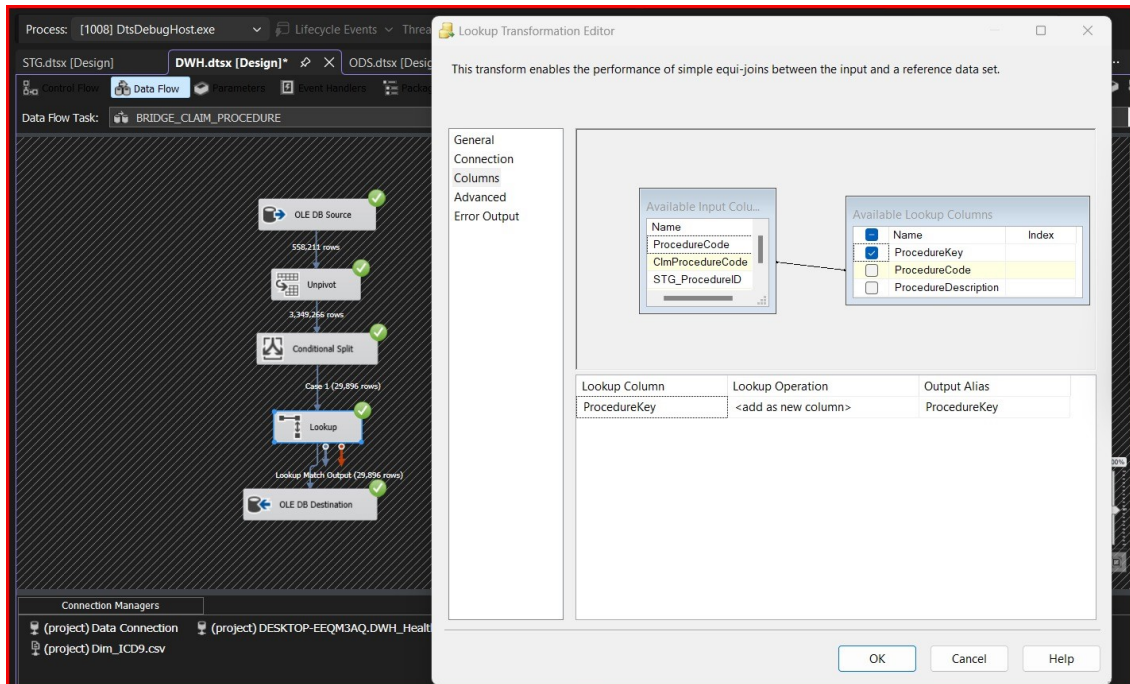


Figure 17: Lookup Transformation Editor for BRIDGE_CLAIM_PROCEDURE - resolving ProcedureCode to ProcedureKey before loading into the bridge table.

10.7. Tabular Model & DAX Semantic Layer

The Power BI semantic model was built on top of FACT_CLAIMS and its related dimensions inside Model.bim (Tabular Model). All business logic is centralized in DAX measures rather than calculated columns, keeping the Fact table lightweight and the calculations fully dynamic.

The screenshot shows the Power BI Model Explorer for 'Model.bim'. The main pane displays the 'FACT_CLAIMS' table with columns: ClaimStart_Date, ClaimEnd_Date, Admission_Date, Discharge_Date, Birth_Date, Death_Date, InrClaimAmtReimbursed, DeductibleAmtPaid, and NoOfMonths_PartA_Cov. The right pane shows a list of DAX measures, including: Cancer Patients, COPD Patients, Depression Patients, Diabetes Patients, Distinct Count of BeneID, Distinct Count of DiagnosisKey, Fraud Claims, Fraud Patients, Fraud Providers, Fraud Rate, Fraud Reimbursed, Heart Failure Patients, Inpatient Claims, Inpatient cost, Inpatient Deductible, Inpatient Patients, Ischemic Heart Patients, Kidney Disease Patients, Non Fraud Claims, Non Fraud Patients, Non Fraud Providers, Osteoporosis Patients, Outpatient Claims, Outpatient cost, Outpatient Patients, Renal Disease Patients, Rheumatoid Arthritis Patients, Stroke Patients, Total Claims, Total Deductible, Total Outpatient Deductible, Total Reimbursed, and Perspectives.

Figure 18: Model.bim Tabular Model Explorer, showing FACT_CLAIMS columns alongside the full list of defined DAX measures (Cancer Patients, COPD Patients, Diabetes Patients, Fraud Claims, Fraud Rate, Inpatient Claims, Total Claims, Total Reimbursed, etc.).

Chronic Condition Measures

Cancer Patients :=

```
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[ChronicCond_Cancer]="YES"
)
```

COPD Patients :=

```
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[ChronicCond_ObstrPulmonary]="YES"
)
```

Depression Patients :=

```
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[ChronicCond_Depression]="YES"
)
```

Diabetes Patients :=

```
CALCULATE(
```

```
        DISTINCTCOUNT([BeneID]),
        DIM_PATIENT[ChronicCond_Diabetes]="YES"
    )

Heart Failure Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[ChronicCond_Heartfailure]="YES"
)

Ischemic Heart Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[ChronicCond_IschemicHeart]="YES"
)

Kidney Disease Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[ChronicCond_KidneyDisease]="YES"
)

Osteoporosis Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[ChronicCond_Osteoporosis]="YES"
)

Rheumatoid Arthritis Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[ChronicCond_Rheumatoidarthritis]="YES"
)

Stroke Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[ChronicCond_stroke]="YES"
)

Renal Disease Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_PATIENT[RenalDiseaseIndicator]="YES"
)
```

Inpatient / Outpatient Measures

```

Inpatient Claims :=
CALCULATE(
    COUNT([ClaimID]),
    DIM_FRAUD[Claim_Type]="IP"
)

Outpatient Claims :=
CALCULATE(
    COUNT([ClaimID]),
    DIM_FRAUD[Claim_Type]="oP"
)

Inpatient Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_FRAUD[Claim_Type]="IP"
)

Outpatient Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_FRAUD[Claim_Type]="oP"
)

Inpatient cost :=
CALCULATE(
    SUM([InscClaimAmtReimbursed]),
    DIM_FRAUD[Claim_Type]="IP"
)

Outpatient cost :=
CALCULATE(
    SUM([InscClaimAmtReimbursed]),
    DIM_FRAUD[Claim_Type]="oP"
)

Inpatient Deductible :=
CALCULATE(
    SUM([DeductibleAmtPaid]),
    DIM_FRAUD[Claim_Type]="IP"
)

Total Outpatient Deductible :=
CALCULATE(
    SUM([DeductibleAmtPaid]),

```

```

    DIM_FRAUD[Claim_Type]="oP"
)

```

```

Average Inpatient Cost :=
DIVIDE(
    [Inpatient cost],
    [Inpatient Claims]
)

```

```

Average Outpatient Cost :=
DIVIDE(
    [Outpatient cost],
    [Outpatient Claims]
)

```

Fraud Measures

```

Fraud Claims :=
CALCULATE(
    COUNT([ClaimID]),
    DIM_FRAUD[PotentialFraud]="Yes"
)

```

```

Non Fraud Claims :=
CALCULATE(
    COUNT([ClaimID]),
    DIM_FRAUD[PotentialFraud]="No"
)

```

```

Fraud Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_FRAUD[PotentialFraud]="Yes"
)

```

```

Non Fraud Patients :=
CALCULATE(
    DISTINCTCOUNT([BeneID]),
    DIM_FRAUD[PotentialFraud]="No"
)

```

```

Fraud Providers :=
CALCULATE(
    DISTINCTCOUNT([Provider]),
    DIM_FRAUD[PotentialFraud]="Yes"
)

```

```

Non Fraud Providers :=
CALCULATE(
    DISTINCTCOUNT([Provider]),
    DIM_FRAUD[PotentialFraud]="No"
)

Fraud Reimbursed :=
CALCULATE(
    SUM([InscClaimAmtReimbursed]),
    DIM_FRAUD[PotentialFraud]="Yes"
)

Fraud Rate :=
DIVIDE(
    [Fraud Claims],
    [Total Claims]
)

```

Base Measures

```

Total Claims :=
COUNT([ClaimID])

Total Deductible :=
SUM([DeductibleAmtPaid])

Total Reimbursed :=
SUM([InscClaimAmtReimbursed])

Distinct Count of BeneID :=
DISTINCTCOUNT([BeneID])

Distinct Count of DiagnosisKey :=
DISTINCTCOUNT([DiagnosisKey])

```

Key DAX Measures - Plain-Language Definitions

Measure	Definition
Total Claims	Total number of claims in the system, indicating overall volume of work.
Distinct Count of BeneID	Number of unique patients benefiting from the service.
Total Reimbursed	Total amount covered and paid out by the insurance company.
Total Deductible	Total amount paid out of pocket by patients.

10.8. Database Schema Diagram

The diagram below shows the final star-schema relationships in the DWH, with FACT_CLAIMS at the center connected to DIM_PROCEDURE, DIM_DIAGNOSIS, DIM_DATE, DIM_FRAUD, DIM_GEN_RAC, DIM_PATIENT, DIM_PHYSICIAN, and DIM_LOCATION, plus the BRIDGE_CLAIM_PROCEDURE and BRIDGE_CLAIM_DIAGNOSIS bridge tables resolving the many-to-many relationships.

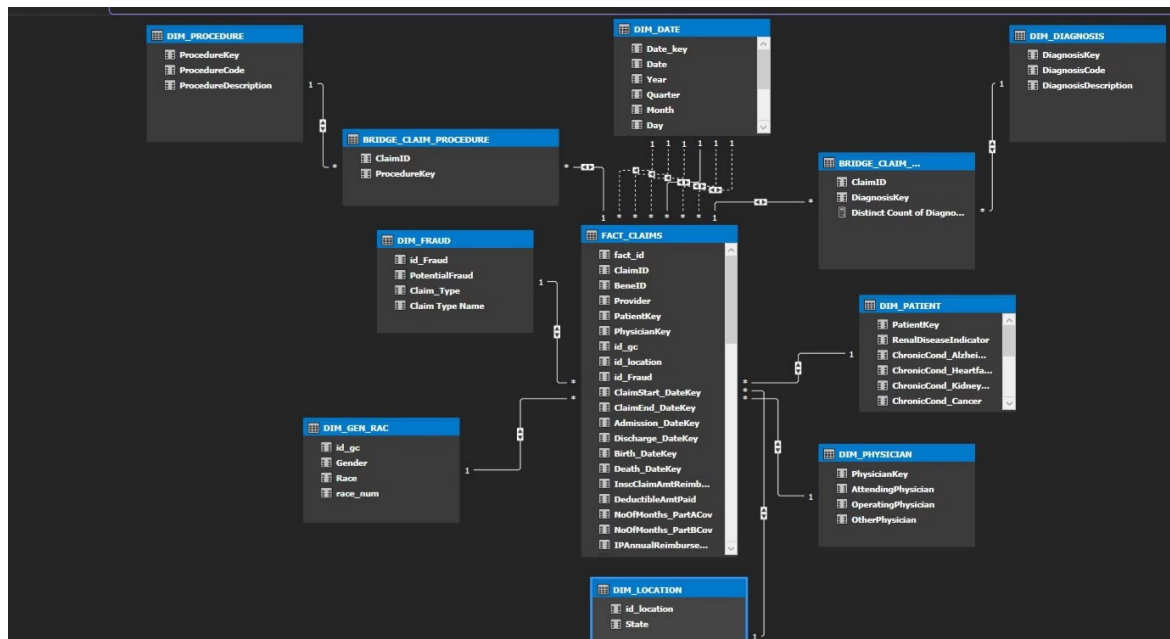


Figure 19: Power BI / Tabular model relationship diagram showing the complete star schema with all dimension, bridge, and fact tables.

10.9. SQL Server Object Explorer

The screenshots below confirm the final physical tables created in each database environment.

DWH_Healthcare Database

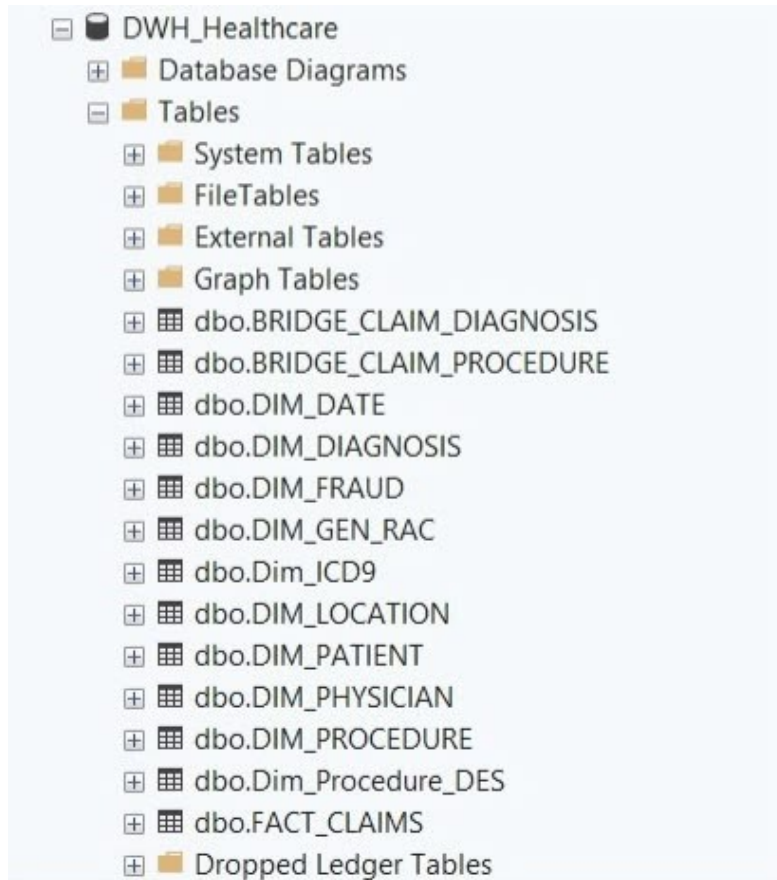


Figure 20: Object Explorer - DWH_Healthcare > Tables, showing BRIDGE_CLAIM_DIAGNOSIS, BRIDGE_CLAIM_PROCEDURE, DIM_DATE, DIM_DIAGNOSIS, DIM_FRAUD, DIM_GEN_RAC, Dim_ICD9, DIM_LOCATION, DIM_PATIENT, DIM_PHYSICIAN, DIM_PROCEDURE, Dim_Procedure_DES, and FACT_CLAIMS.

ODS_Project Database



Figure 21: Object Explorer - ODS_Project > Tables, showing dbo.ODS_Healthcare_All_Claims, the consolidated Operational Data Store table feeding the downstream STG/DWH packages.

10.10. Power BI Dashboard

The following pages show the final Power BI report built on top of the Data Warehouse model: a cover page, an executive overview, descriptive analysis, inpatient analysis, outpatient analysis, and a dedicated fraud detection page.

Cover Page



Figure 26: Report cover page - "Healthcare Provider Fraud Detection Analysis: An End-to-End Data Engineering Solution for Healthcare Fraud Detection", with navigation to Overview, Inpatient Analysis, Descriptive Analysis, Outpatient Analysis, and Fraud Detection.

Overview Page

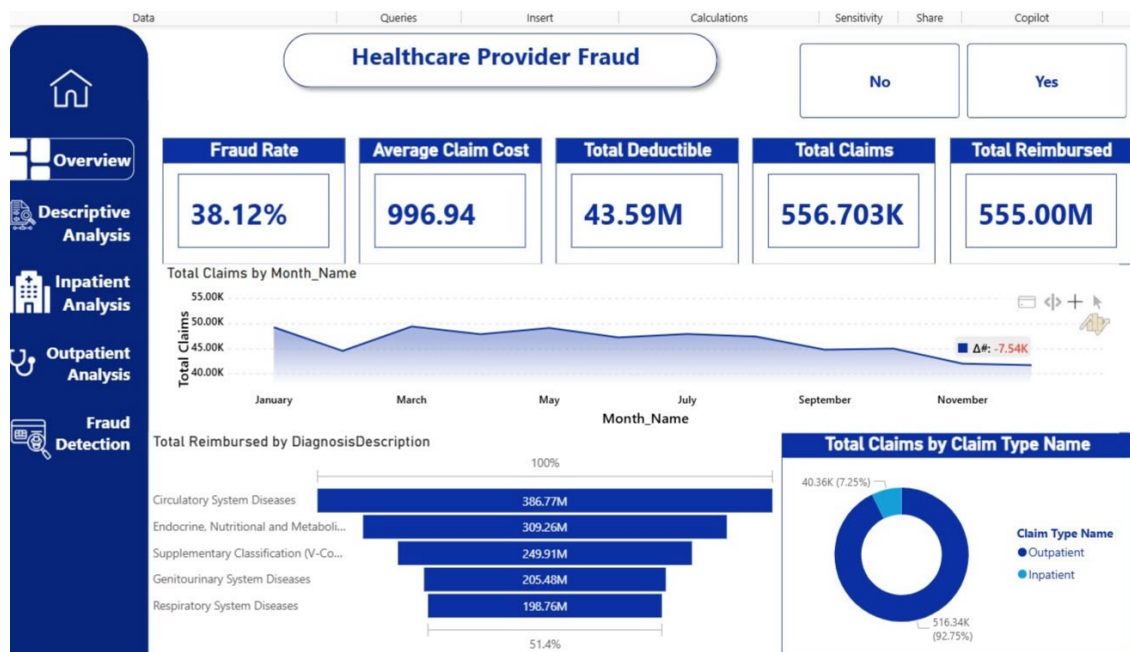


Figure 27: Overview page - Fraud Rate (38.12%), Average Claim Cost (996.94), Total Deductible (43.59M), Total Claims (556.703K), Total Reimbursed (555.00M), Total Claims by Month_Name, Total Reimbursed by DiagnosisDescription, and Total Claims by Claim Type Name (Outpatient vs Inpatient).

Descriptive Analysis Page

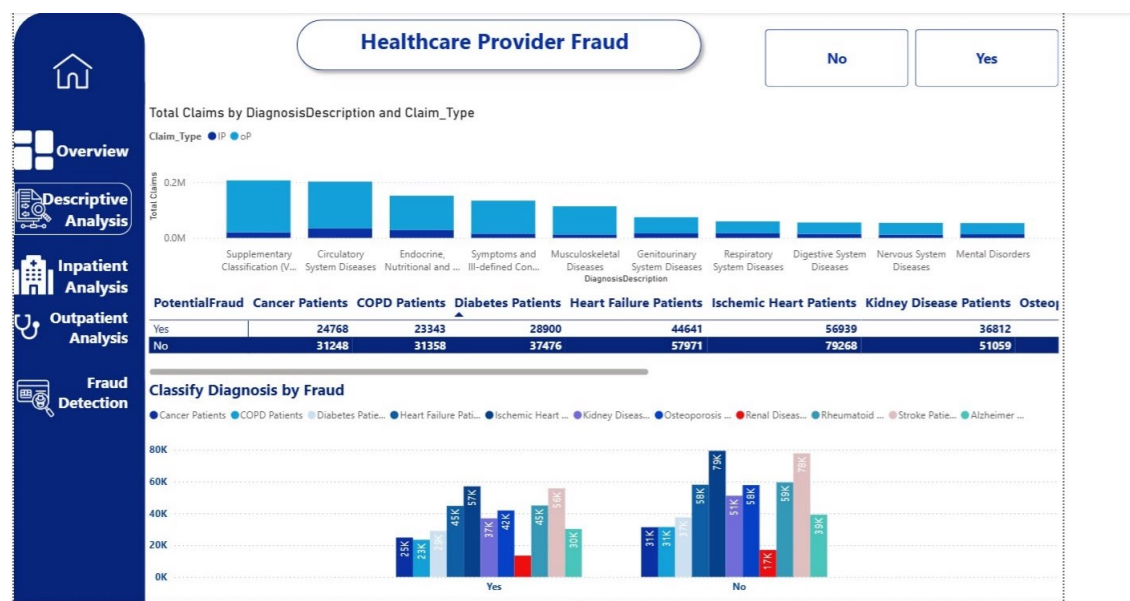


Figure 24: Descriptive Analysis page - Total Claims by DiagnosisDescription and Claim_Type, a PotentialFraud breakdown matrix by chronic condition (Cancer, COPD, Diabetes, Heart Failure, Ischemic Heart, Kidney Disease, Osteoporosis...), and a Classify Diagnosis by Fraud clustered column chart (Yes/No).

Inpatient Analysis Page

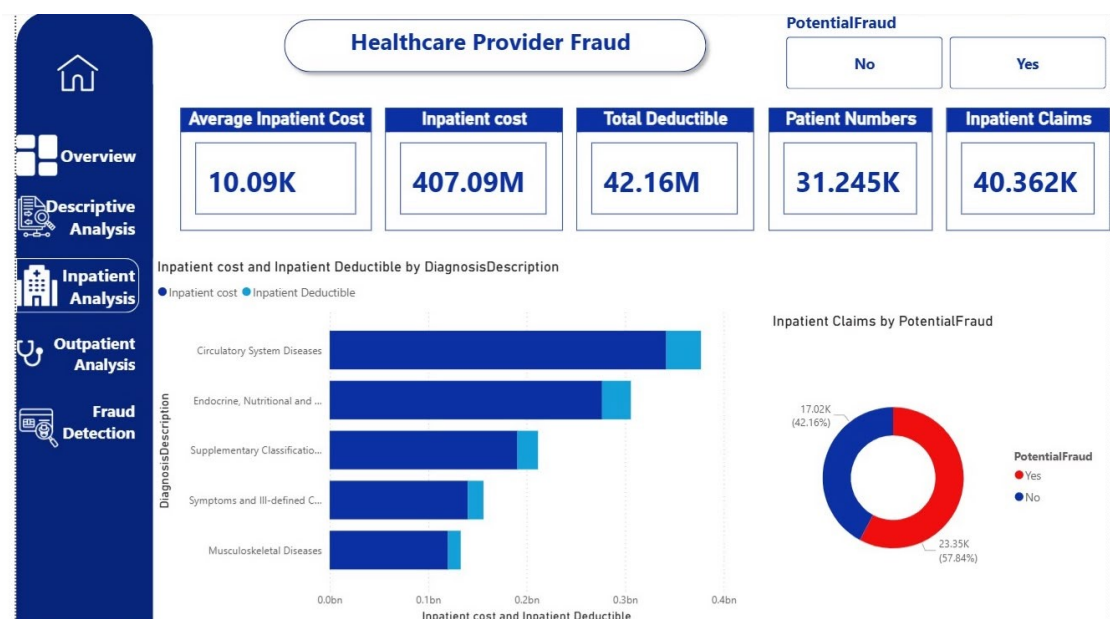


Figure 25: Inpatient Analysis page - Average Inpatient Cost (10.09K), Inpatient cost (407.09M), Total Deductible (42.16M), Patient Numbers (31.245K), Inpatient Claims (40.362K), Inpatient cost and Deductible by DiagnosisDescription, and Inpatient Claims by PotentialFraud donut chart.

Outpatient Analysis Page

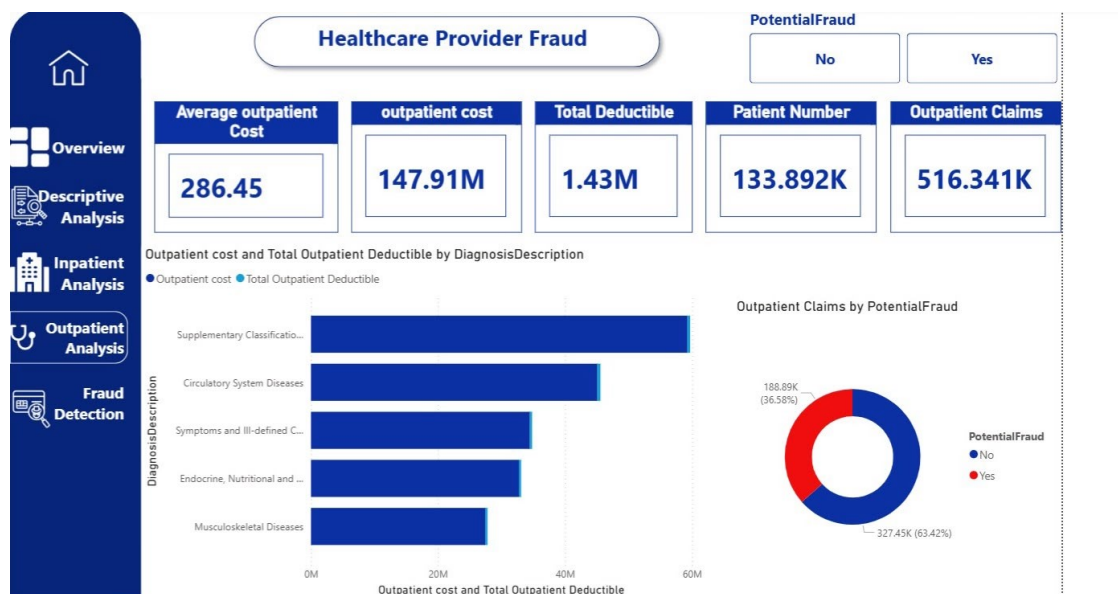


Figure 26: Outpatient Analysis page - Average outpatient Cost (286.45), outpatient cost (147.91M), Total Deductible (1.43M), Patient Number (133.892K), Outpatient Claims (516.341K), Outpatient cost and Deductible by DiagnosisDescription, and Outpatient Claims by PotentialFraud donut chart.

Fraud Detection Page

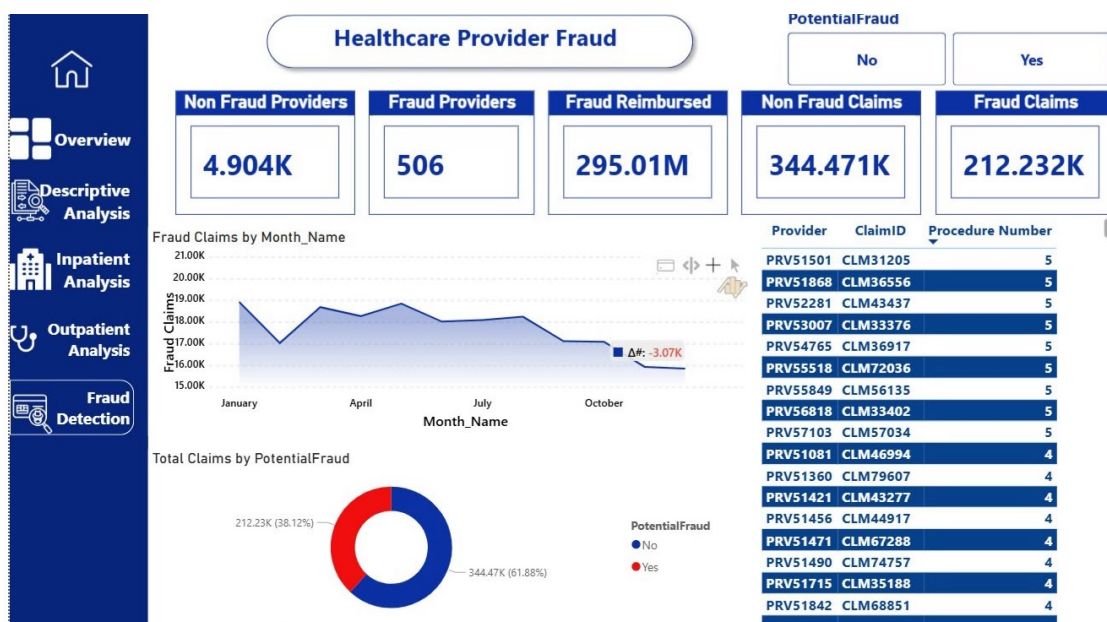


Figure 27: Fraud Detection page - Non Fraud Providers (4.904K), Fraud Providers (506), Fraud Reimbursed (295.01M), Non Fraud Claims (344.471K), Fraud Claims (212.232K), Fraud Claims by Month_Name trend line, Total Claims by PotentialFraud donut chart, and a detailed Provider / ClaimID / Procedure Number table.

11. n8n for project Overview

11.1 What the System Does

This project is an end-to-end intelligent system for detecting potential healthcare insurance fraud. It collects patient and claim data interactively through a Telegram chatbot, feeds the data to a trained machine learning model via a REST API, and returns a fraud prediction result — all automated through an n8n workflow.

11.2 Problem It Solves

Healthcare fraud is a major global challenge. Fraudulent insurance claims cost billions of dollars annually and divert resources from legitimate patients. Manual review of every claim is expensive and slow. This system automates the initial fraud screening process by:

- Collecting structured patient and claim data from medical staff via Telegram
- Running the data through a Gradient Boosting classifier trained on real Medicare data
- Returning an instant fraud probability score to the user
- Allowing reviewers to focus manual effort on flagged high-risk cases

11.3 System Components

Component	Technology	Role
ML Model	Gradient Boosting (scikit-learn)	Predicts fraud probability
REST API	Flask + ngrok	Serves model predictions over HTTP
Automation	n8n Workflow	Orchestrates data collection and routing
Interface	Telegram Bot	User-facing chatbot for data input & results
Storage	n8n Data Table	Persists session state across messages

11.4. Dataset & Features

11.4.1 Data Source

The dataset used for training is the

Healthcare Provider Fraud Detection Analysis dataset from Kaggle (rohitrox/healthcare-provider-fraud-detection-analysis). It contains real Medicare claims data across four files:

- Train.csv — Provider-level fraud labels (Yes/No)
- Train_Beneficiarydata — Patient demographics and chronic conditions
- Train_Inpatientdata — Inpatient hospital claims
- Train_Outpatientdata — Outpatient clinic claims

All four files were merged on shared keys (BenefID, ClaimID, Provider) to create a unified dataset of over 650,000 records.

11.4.2 Target Variable

PotentialFraud — Binary label indicating whether a provider is potentially fraudulent.

- 1 = Fraud (positive class)
- 0 = Not Fraud (negative class)

11.4.3 Class Imbalance

The original training data was imbalanced. SMOTE (Synthetic Minority Oversampling Technique) was applied to balance the classes before training:

Dataset	Not Fraud (0)	Fraud (1)	Total
Before SMOTE	241,790	148,957	390,747
After SMOTE	241,790	241,790	483,580
Test Set	103,625	63,839	167,464

11.4.4 The 12 Features Used in the Bot

After feature selection using Mutual Information scoring and exploratory analysis, 12 key features were selected for the Telegram bot interface:

#	Feature Name (API)	Bot Question	Type
1	ChronicCond_Diabetes	Does the patient have diabetes?	Binary (0/1)

2	ChronicCond_IschemicHeart	Does the patient have ischemic heart disease?	Binary (0/1)
3	NoOfMonths_PartACov	Months covered by Medicare Part A?	Numeric (0–12)
4	ChronicCond_Heartfailure	Does the patient have heart failure?	Binary (0/1)
5	Gender	Patient's sex?	Binary (1=Male, 2=Female)
6	NoOfMonths_PartBCov	Months covered by Medicare Part B?	Numeric (0–12)
7	State	Patient's state?	Categorical (encoded)
8	Race	Patient's race/ethnicity?	Categorical (encoded)
9	DeductibleAmtPaid	Total deductible paid?	Numeric (\$)
10	InscClaimAmtReimbursed	Insurance claim reimbursed?	Numeric (\$)
11	IPAnnualDeductibleAmt	Annual inpatient deductible?	Numeric (\$)
12	IPAnnualReimbursementAmt	Annual inpatient reimbursement?	Numeric (\$)

11.5. Machine Learning Model

11.5.1 Algorithm

The final model is a **Gradient Boosting Classifier** from scikit-learn. Gradient Boosting builds an ensemble of decision trees sequentially, where each tree corrects the errors of the previous one, making it highly effective for tabular, structured data.

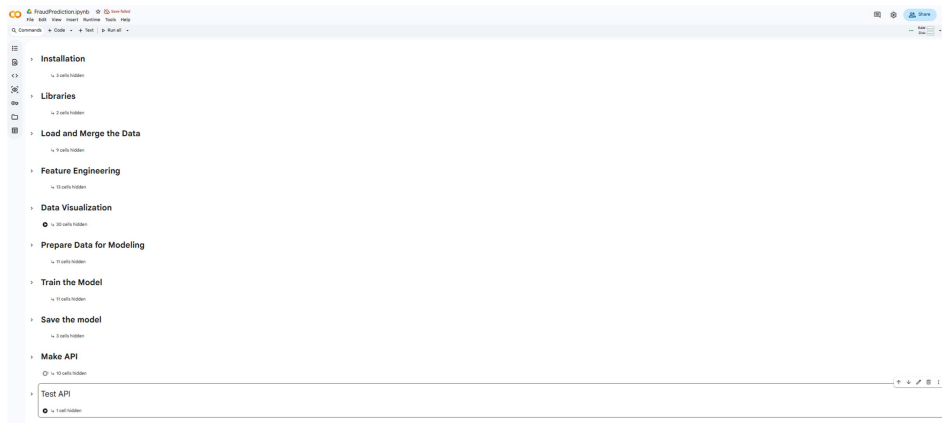
Hyperparameters used:

Parameter	Value	Effect
learning_rate	0.2	Step size for each boosting iteration
max_depth	7	Maximum depth of individual trees
n_estimators	50	Number of boosting stages
random_state	42	Reproducibility seed

11.5.2 Training Pipeline

The model was trained in Google Colab using a GPU (T4). The full pipeline:

- Step 1 — Data Loading: Merged 4 CSV files into a single DataFrame
- Step 2 — Preprocessing: Encoded chronic conditions (2→0), label-encoded categorical columns, dropped high-cardinality ID columns
- Step 3 — Feature Engineering: Computed phy_count (number of physicians per claim) and period (claim duration in days)
- Step 4 — Feature Selection: Used Mutual Information scoring to rank features; selected top 24 features
- Step 5 — Train/Test Split: 70% training, 30% test, stratified by target
- Step 6 — Scaling: Applied MinMaxScaler on training data; transformed test data
- Step 7 — SMOTE: Oversampled minority (Fraud) class to balance training set
- Step 8 — Training: Fit GradientBoostingClassifier on balanced data
- Step 9 — Saving: Exported model as fraud_model.keras, scaler as scaler.pkl, feature list as features.pkl



11.5.3 Performance Metrics

The model was evaluated on the held-out test set (167,464 records):

Metric	Class 0 (Not Fraud)	Class 1 (Fraud)	Overall
Precision	92%	88%	—
Recall	93%	87%	—
F1-Score	92%	88%	—
Accuracy	—	—	90.59%
Train Accuracy	—	—	91.60%

... Accuracy: 90.59%

Confusion Matrix:

```
[[96123  7502]
 [ 8260 55579]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.92	0.93	0.92	103625
1	0.88	0.87	0.88	63839
accuracy			0.91	167464
macro avg	0.90	0.90	0.90	167464
weighted avg	0.91	0.91	0.91	167464

11.5.4 Confusion Matrix

	Predicted: Not Fraud	Predicted: Fraud
Actual: Not Fraud	96,123 (TN)	7,502 (FP)
Actual: Fraud	8,261 (FN)	55,578 (TP)

11.5.5 Saved Model Files

File	Description
fraud_model.keras	Trained Gradient Boosting model serialized for TensorFlow/Keras compatibility
scaler.pkl	Fitted MinMaxScaler for normalizing input features
features.pkl	Ordered list of feature names expected by the model

11.6. API Development

11.6.1 Framework

The prediction API is built with Flask, a lightweight Python web framework. It runs inside Google Colab and is exposed to the internet via ngrok, which creates a secure tunnel from the public internet to the local Colab runtime.

Tool	Version	Role
Flask	3.1.3	Web framework for the REST API
pyngrok	8.1.2	ngrok Python wrapper for tunnel creation
TensorFlow/Keras	Latest	Loading the saved .keras model
Joblib	1.5.3	Loading scaler.pkl and features.pkl
pandas	Latest	Building DataFrames from request JSON

11.6.2 Endpoints

POST /predict

The main prediction endpoint. Accepts a JSON body with 12 feature values and returns a fraud prediction.

Request Body (JSON):

```
{
  "ChronicCond_Diabetes": 1,
  "ChronicCond_IschemicHeart": 0,
  "NoOfMonths_PartACov": 12,
  "ChronicCond_Heartfailure": 0,
  "Gender": 1,
  "NoOfMonths_PartBCov": 12,
  "State": 5,
  "Race": 1,
  "DeductibleAmtPaid": 500,
  "InscClaimAmtReimbursed": 3000,
  "IPAnnualDeductibleAmt": 1200,
  "IPAnnualReimbursementAmt": 25000
}
```

Response (JSON):


```
{
  "prediction": 0,
  "probability": 0.0312,
  "result": "Not Fraud"
}
```

Field	Type	Description
prediction	int (0 or 1)	0 = Not Fraud, 1 = Fraud
probability	float (0.0–1.0)	Model confidence score (threshold: 0.5)
Result	string	Human-readable label: 'Fraud' or 'Not Fraud'

GET /

Health check endpoint. Returns API status and available endpoints.

11.6.3 Internal Processing Logic

When a POST request hits /predict, the API performs these steps:

- Parse the JSON body from the request
- Convert to a pandas DataFrame with one row
- Reindex columns to match the order in features.pkl (fills missing columns with 0)
- Apply the fitted MinMaxScaler to normalize values
- Run model.predict() to get a probability score
- Apply threshold (≥ 0.5 = Fraud) to produce binary prediction
- Return JSON response with prediction, probability, and result label

11.6.4 Running the API (Google Colab)

- Upload fraud_model.keras, scaler.pkl, and features.pkl to your Colab session
- Run Run_Api.ipynb cell by cell
- The Flask app starts on port 5000 in a background thread
- ngrok creates a public tunnel and prints the URL
- Copy the URL and update the HTTP Request node in n8n

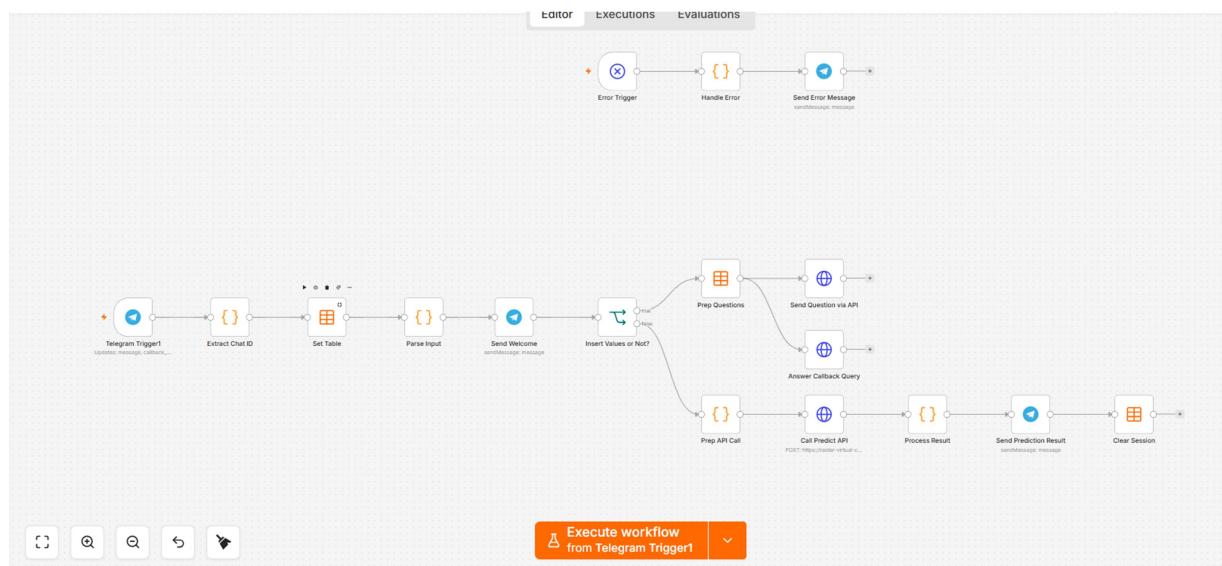
The ngrok public URL changes every session unless you have a paid ngrok plan with a static domain.

11.7. Workflow Automation (n8n)

11.7.1 What is n8n?

n8n is an open-source, self-hostable workflow automation platform. It connects different services and APIs using a visual node-based editor. Each 'node' in n8n performs a specific action (send a message, query a database, call an API, run JavaScript code, etc.), and nodes are connected to form an automated pipeline.

11.7.2 Workflow Architecture



The workflow consists of two branches diverging from a single decision node:

Branch	Trigger Condition	Flow
Continue (True)	action = 'upsert' — survey in progress	Save Session → Send Next Question
Done (False)	action = 'done' — all 12 answers collected	Build Payload → Call API → Send Result → Delete Session

An error-handling sub-workflow (Error Trigger → Handle Error → Send Error Message) runs if any node fails, notifying the user via Telegram.

11.7.3 Node-by-Node Explanation

Node 1: Telegram Trigger

- Type: Telegram Trigger
- Listens for: message and callback_query events
- Fires every time the user sends a message to the bot
- Extracts: chat.id, message.text

Node 2: Get Session (Set Table / Data Table)

- Type: Data Table — Get operation
- Table: medical_sessions
- Filters by: chat_id = current user's chat ID
- Purpose: Retrieve the saved survey state (step number + answers so far)
- alwaysOutputData: true — ensures the flow continues even if no session exists yet

Node 3: Parse Input / Process Flow

- Type: Code (JavaScript) — runs once per item
- Purpose: Core session logic — determines what to do with the incoming message
- Logic:
- If /start or no session exists → initialize new session at step 1
- Otherwise → save the current answer, advance step counter
- If step > 12 → set action = 'done' with all 12 answers
- Otherwise → set action = 'upsert' with next question text

Node 4: Insert Values or Not? (Continue or Done?)

- Type: IF condition node
- Condition: action == 'upsert'
- True output (branch 1) → Save Session → Send Question
- False output (branch 2) → Build Payload → Call API → Send Result

Node 5a: Save Session

- Type: Data Table — Upsert operation
- Table: medical_sessions
- Matches on: chat_id
- Saves: step, and all 12 answer fields
- Upsert: creates the row if it doesn't exist, updates it if it does

Node 5b: Send Welcome / Send Questions

- Type: Telegram — Send Message
- Sends: the question text from Process Flow (Markdown formatted)
- Uses parse_mode: Markdown for bold text and code formatting

Node 6: Prep API Call / Build Payload

- Type: Code (JavaScript) — Run Once for All Items
- Purpose: Converts raw text answers into numeric values for the model
- Conversions performed:
- Yes/No → 1/0 for chronic conditions
- Male/Female → 1/2 for Gender
- Dollar amounts → parseFloat()
- Missing values → 0 (default)
- Output: { chatId, payload: { ...12 features } }

Node 7: Call Predict API / HTTP Request

- Type: HTTP Request
- Method: POST
- URL: https://<ngrok-domain>.ngrok-free.app/predict
- Body: \$json.payload (the 12 numeric features)
- Returns: { prediction, probability, result }

Node 8: Process Result / Format Result

- Type: Code (JavaScript)
- Formats the API response into a user-friendly Telegram message
- If prediction = 1: Red circle emoji, 'Potential Fraud Detected', probability %
- If prediction = 0: Green circle emoji, 'No Fraud Indicators', probability %

Node 9: Send Prediction Result

- Type: Telegram — Send Message
- Sends the formatted result to the user
- Uses parse_mode: Markdown for bold text

Node 10: Clear Session / Delete Session

- Type: Data Table — Delete operation
- Removes the user's row from medical_sessions after the survey is complete
- Allows the user to run /start again for a new prediction

11.8. Telegram Bot

11.8.1 How the Bot Works

The Telegram Bot is the user interface for this system. It acts as a conversational form — asking 12 questions one at a time and collecting the answers needed by the fraud detection model. The bot is powered by the n8n workflow running in the background.



11.8.2 User Journey

Step	User Action	Bot Response
1	Sends /start	Welcome message + Question 1/12
2–12	Answers each question	Next question in sequence
13	Answers Question 12/12	Calls API → Returns fraud prediction result
14	Sends /start again	Resets session, starts new survey

11.8.3 Question Format

Each question is formatted in Markdown with:

- Question number and total (e.g. Question 3/12)
- Bold field name for clarity
- Expected answer format (Yes/No, number range, or list of options)
- Emoji icons to make the conversation visually friendly

11.8.4 Result Format

After the 12th answer, the bot returns one of two possible results:

Prediction	Message Shown to User
Fraud (1)	☐ Result: Potential Fraud Detected\nProbability Score: XX%\nRecommended: Manual review
Not Fraud (0)	☐ Result: No Fraud Indicators Detected\nProbability Score: XX%\nThe claim appears legitimate

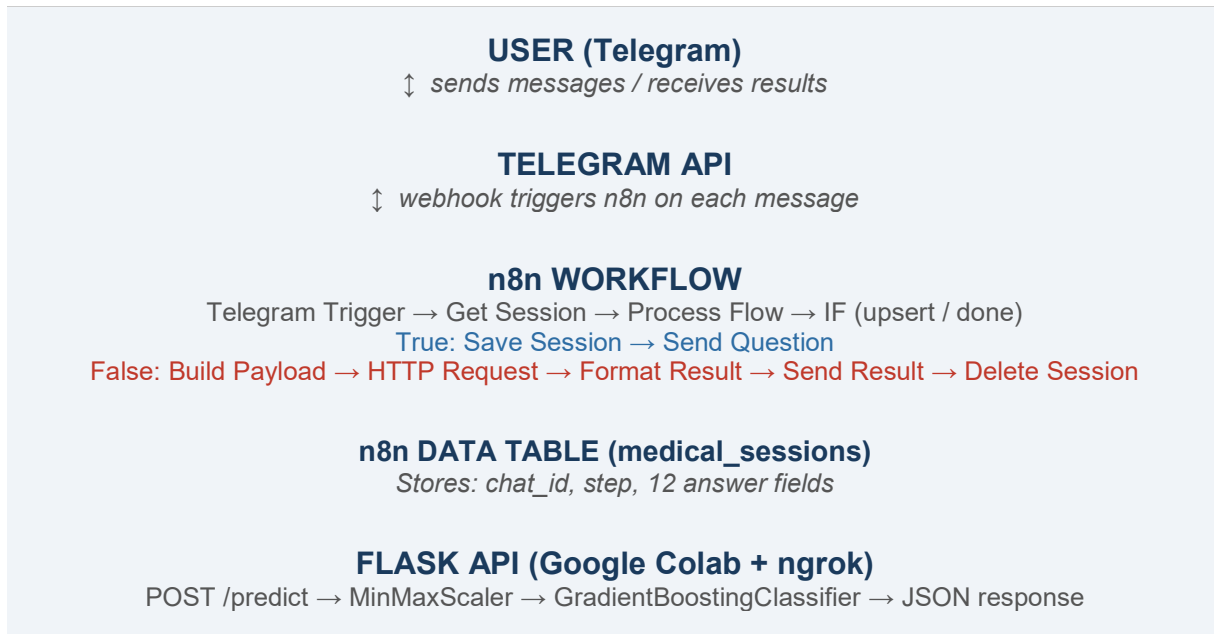
11.8.5 Session Management

Each user's survey state is stored in the n8n Data Table (medical_sessions). This allows the bot to:

- Resume a conversation if the user takes a break between questions
- Handle multiple users simultaneously (each identified by their unique chat_id)
- Clean up the session automatically after the survey completes

11.9 System Architecture

The complete end-to-end data flow is illustrated below:



11.10. Results & Discussion

11.10.1 Model Performance Summary

The Gradient Boosting Classifier achieved 90.59% accuracy on the test set, with an F1-score of 0.92 for the Not Fraud class and 0.88 for the Fraud class. The small gap between training accuracy (91.60%) and test accuracy (90.59%) indicates the model generalizes well without significant overfitting.

11.10.2 Handling Class Imbalance

The original dataset had approximately a 62/38 split between Not Fraud and Fraud cases. SMOTE was used to oversample the Fraud class to achieve a 50/50 split before training. This improved recall for the Fraud class from what would have been a biased model to 87%, meaning the model correctly identifies 87 out of every 100 fraudulent cases.

11.10.3 Feature Importance Insights

Mutual Information analysis revealed that Provider ID was by far the most informative feature (MI Score: 0.664). Among the clinically meaningful features available through the bot, the most important were:

- ChronicCond_IschemicHeart (MI: 0.064) — strongest clinical predictor
- ChronicCond_Diabetes (MI: 0.064) — highly correlated with fraud patterns
- NoOfMonths_PartACov (MI: 0.059) — coverage duration as a proxy for provider behavior
- ChronicCond_Heartfailure (MI: 0.057) — significant in fraud cases

11.10.4 System Integration

The end-to-end integration between Telegram, n8n, and the Flask API was successfully demonstrated. The bot correctly collects all 12 inputs, the n8n workflow correctly builds and routes the API payload, and the Flask API returns accurate predictions. The complete round trip (user answers last question → prediction received in Telegram) takes approximately 2–4 seconds.

11.11. Future Work

11.11.1 Model Improvements

- Include all 24 features used during training (currently the bot collects only 12)
- Experiment with XGBoost or LightGBM for potentially higher accuracy
- Perform hyperparameter tuning with GridSearchCV or Optuna
- Add model explainability (SHAP values) to show which features drove each prediction

11.11.2 API Improvements

- Deploy the API to a persistent cloud platform (AWS, GCP, Heroku) to eliminate ngrok dependency
- Add API authentication (API key or OAuth) to restrict access
- Add input validation to reject out-of-range values (e.g., months > 12)
- Implement rate limiting and request logging

11.11.3 Bot & Workflow Improvements

- Add inline keyboard buttons for Yes/No questions instead of free text
- Add input validation in the bot (reject non-numeric answers for numeric questions)
- Support multiple languages (Arabic, English)
- Add an admin dashboard node that logs all predictions to a Google Sheet
- Support editing a previous answer without restarting the survey

11.11.4 Deployment

- Host n8n on a VPS (DigitalOcean, Render) for always-on operation
- Use a proper database (PostgreSQL) instead of n8n Data Tables for session storage
- Set up monitoring and alerting for API downtime